

Artificial Intelligence for Science



Lesson 11: Forward Neural Network (前馈神经网络)

CHEN Kejie 陈克杰 (chenkj@sustech.edu.cn)

27-Apr-26



PCA 是线性降维方法，本质是找到一组正交方向，使得数据在这些方向上的方差最大。

数学本质是：将数据投影到协方差矩阵的前几个主特征向量上。

局限性：

- 只能发现 线性结构
- 无法适应数据中的复杂模式（比如非线性流形）

PCA人脸



AutoEncoder



Reducing the Dimensionality of Data with Neural Networks

G. E. HINTON AND R. R. SALAKHUTDINOV [Authors Info & Affiliations](#)

SCIENCE • 28 Jul 2006 • Vol 313, Issue 5786 • pp. 504-507 • DOI: 10.1126/science.1127647

43,019 14,647



Abstract

High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors. Gradient descent can be used for fine-tuning the weights in such “autoencoder” networks, but this works well only if the initial weights are close to a good solution. We describe an effective way of initializing the weights that allows deep autoencoder networks to learn low-dimensional codes that work much better than principal components analysis as a tool to reduce the dimensionality of data.



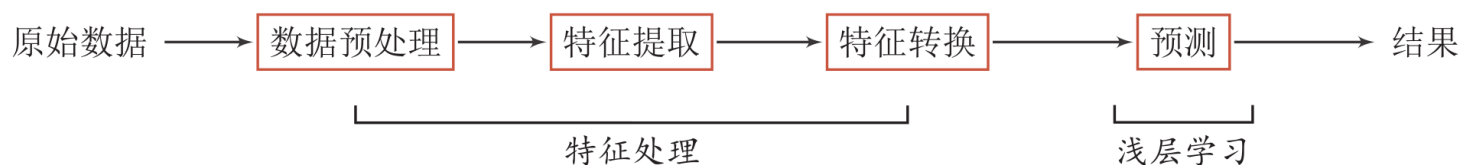
神经网络：非线性降维，引爆深度学习



表示学习

机器学习

当我们用机器学习来解决一些模式识别任务时，一般的流程包含以下几个步骤：



特征工程 (Feature Engineering)

浅层学习 (Shallow Learning) :

不涉及特征学习，其特征主要靠人工经验或特征转换方法来抽取。

语义鸿沟：人工智能的挑战之一

底层特征 VS 高层语义

人们对文本、图像的理解无法从字符串或者图像的底层特征直接获得



床前明月光，
疑是地上霜。
举头望明月，
低头思故乡。

表示学习

Bengio, Yoshua, Aaron Courville, and Pascal Vincent.
"Representation learning: A review and new perspectives."
IEEE transactions on pattern analysis and machine intelligence
35.8 (2013): 1798-1828.

- ▶ 数据表示是机器学习的核心问题。

- ▶ 特征工程：需要借助人脑智能

- ▶ 表示学习

- ▶ 如何自动从数据中学习好的表示

- ▶ 难点

- ▶ 没有明确的目标

什么是好的数据表示？

- ▶“好的表示”是一个非常主观的概念，没有一个明确的标准。
- ▶但一般而言，一个好的表示具有以下几个优点：
 - ▶应该具有很强的表示能力。
 - ▶应该使后续的学习任务变得简单。
 - ▶应该具有一般性，是任务或领域独立的。

表示形式

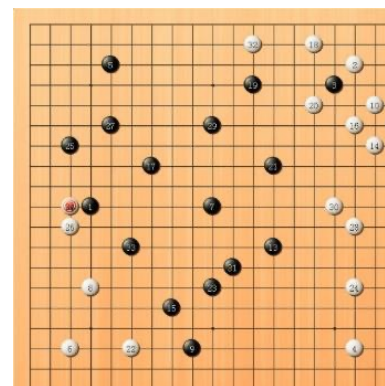
▶ 局部表示

- ▶ 离散表示、符号表示
- ▶ One-Hot向量

	局部表示	分布式表示
A	[1 0 0 0]	[0.25 0.5]
B	[0 1 0 0]	[0.2 0.9]
C	[0 0 1 0]	[0.8 0.2]
D	[0 0 0 1]	[0.9 0.1]

▶ 分布式(distributed)表示

- ▶ 压缩、低维、稠密向量
- ▶ 用 $O(N)$ 个参数表示 $O(2^k)$ 区间
 - ▶ k 为非0参数, $k < N$
 - ▶ 二分, $O(2^k)$
 - ▶ 若采用局部表示, 只能 k 个语义



分布式表示

一个生活中的例子：颜色

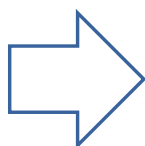
颜色	局部表示	分布式表示
琥珀色	$[1, 0, 0, 0]^T$	$[1.00, 0.75, 0.00]^T$
天蓝色	$[0, 1, 0, 0]^T$	$[0.00, 0.5, 1.00]^T$
中国红	$[0, 0, 1, 0]^T$	$[0.67, 0.22, 0.12]^T$
咖啡色	$[0, 0, 0, 1]^T$	$[0.44, 0.31, 0.22]^T$



语义表示

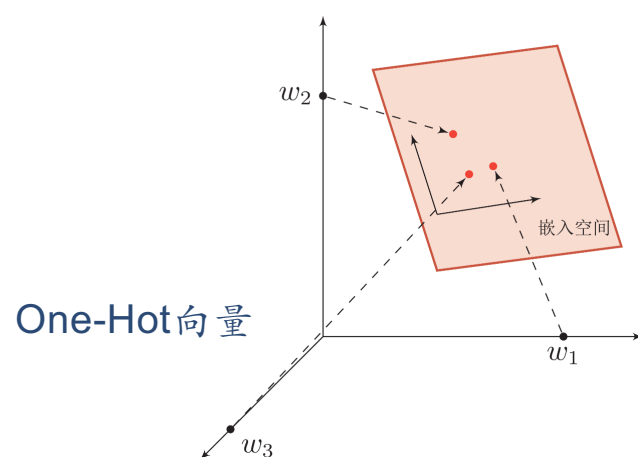
局部（符号）表示

知识库、规则

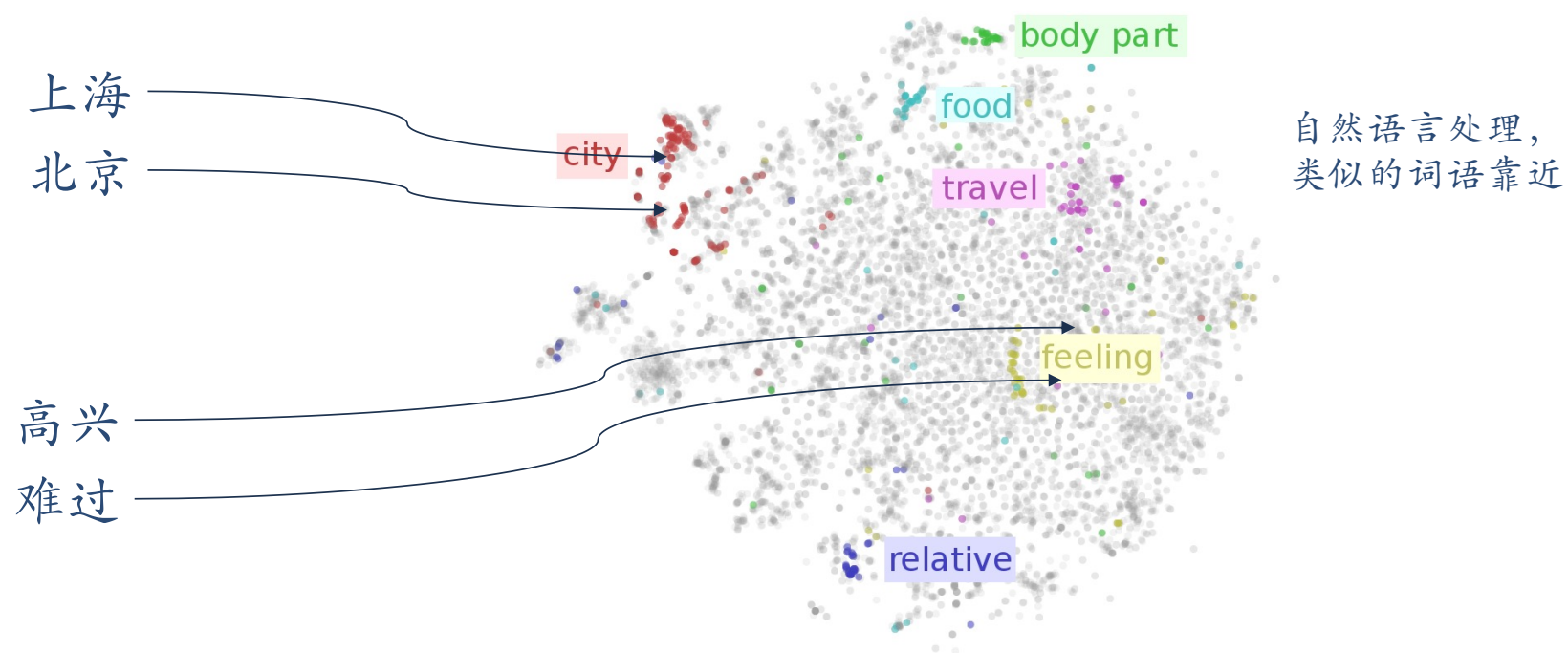


分布式表示

嵌入：压缩、低维、稠密向量



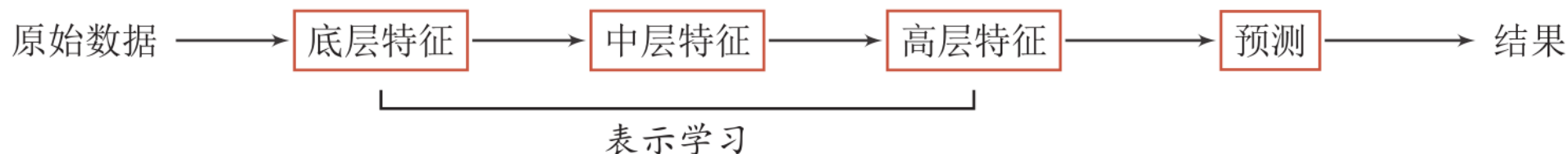
词嵌入 (Word Embeddings)



<https://indico.io/blog/visualizing-with-t-sne/>

表示学习

- ▶ 表示学习:如何自动从数据中学习好的表示
- ▶ 通过构建具有一定“深度”的模型，可以让模型来自动学习好的特征表示（从底层特征，到中层特征，再到高层特征），从而最终提升预测或识别的准确性。



传统的特征提取VS表示学习

▶特征提取

- ▶线性投影（子空间）
 - ▶PCA、LDA
- ▶非线性嵌入
 - ▶LLE、Isomap、谱方法
- ▶自编码器

▶特征提取VS表示学习

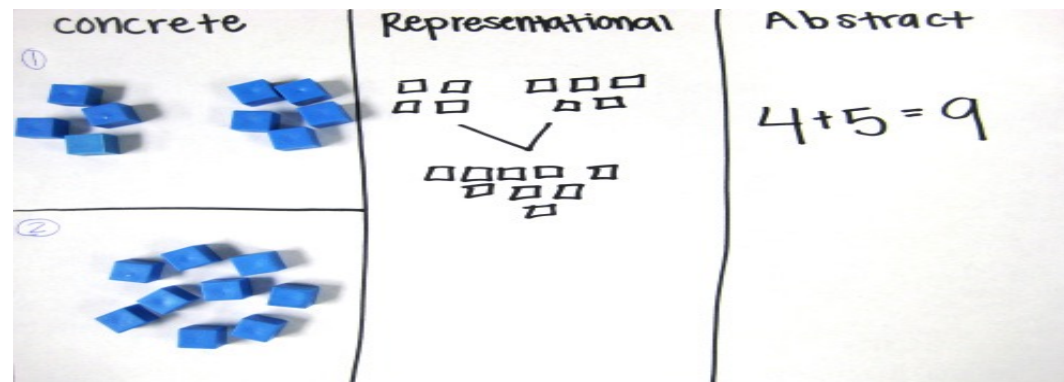
- ▶特征提取：基于任务或先验对去除无用特征，对后面分类可能无意义
- ▶表示学习：通过深度模型学习高层语义特征，难点：没有明确目标



深度学习

表示学习与深度学习

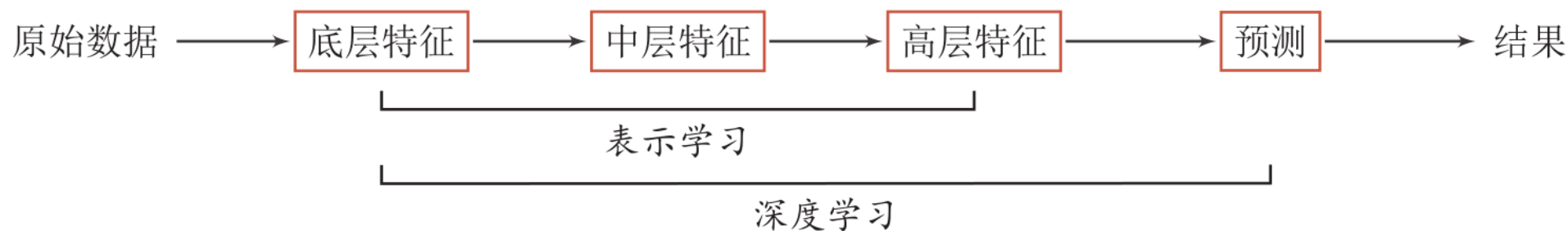
- ▶ 一个好的表示学习策略必须具备一定的深度
 - ▶ 特征重用(一些较早提出的特征仍可被较深层直接使用)
 - ▶ 指数级的表示能力
 - ▶ 抽象表示与不变性
 - ▶ 抽象表示需要多步的构造



<https://mathteachingstrategies.wordpress.com/2008/11/24/concrete-and-abstract-representations-using-mathematical-tools/>

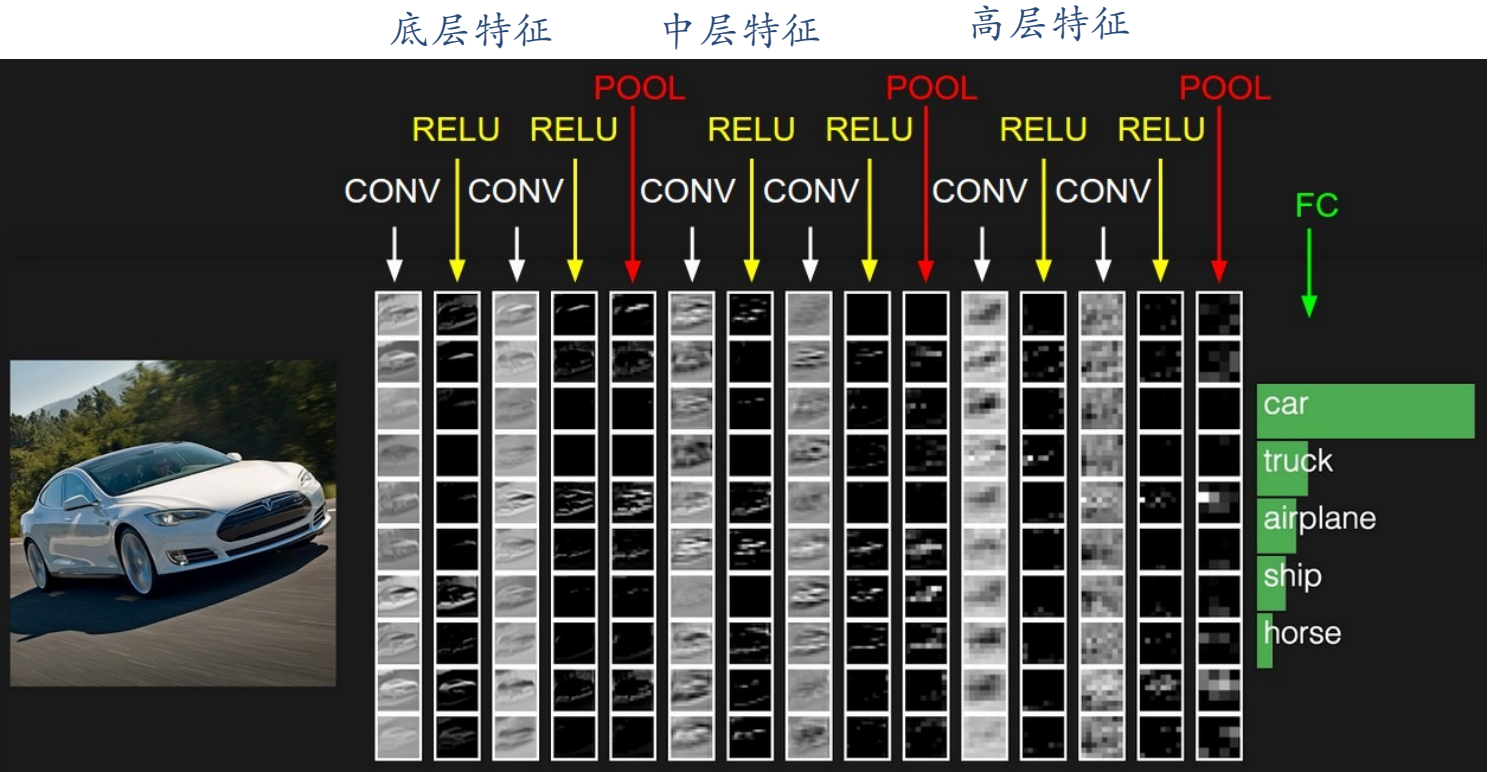
深度学习 (Deep Learning)

► 深度学习 = 表示学习 + 决策学习

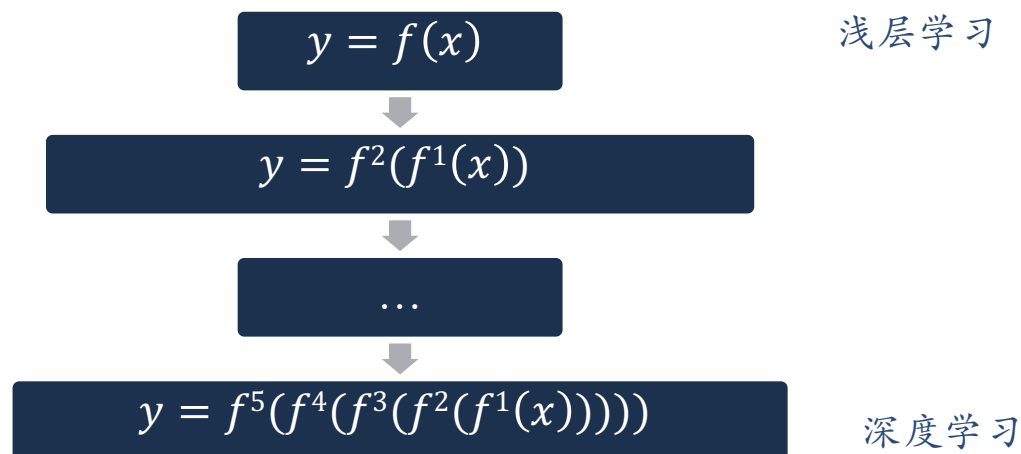


难点：贡献度分配问题

表示学习与深度学习



深度学习的数学描述



$f^l(x)$ 为非线性函数，不一定连续。

当 $f^l(x)$ 连续时，比如 $f^l(x) = \sigma(W^l f^{l-1}(x))$ ，这个复合函数称为神经网络。

神经网络天然不是深度学习,但是深度学习天然神经网络

深度学习的定义：

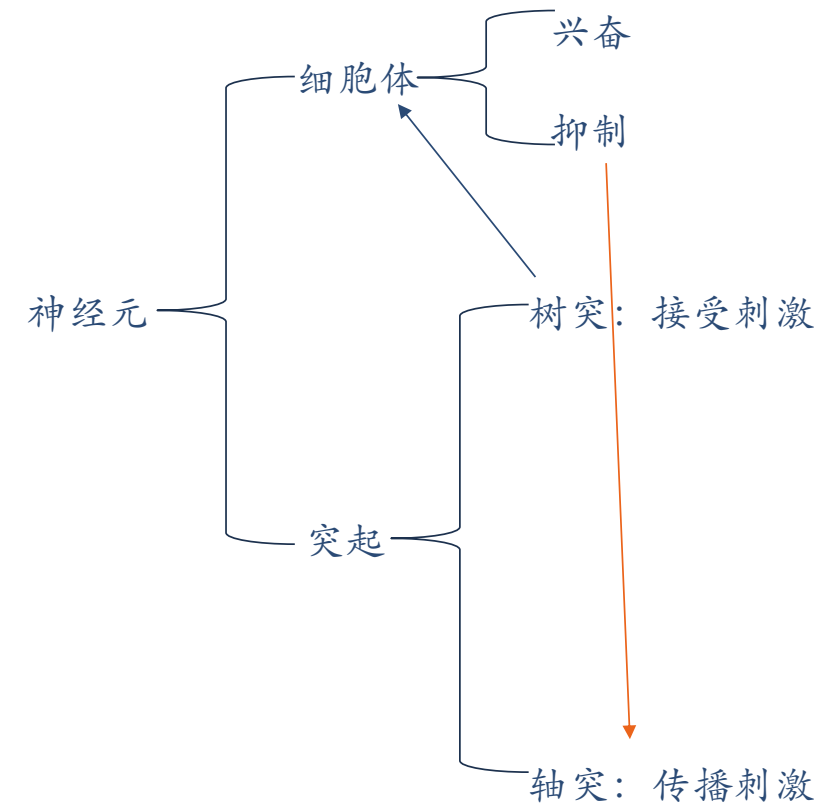
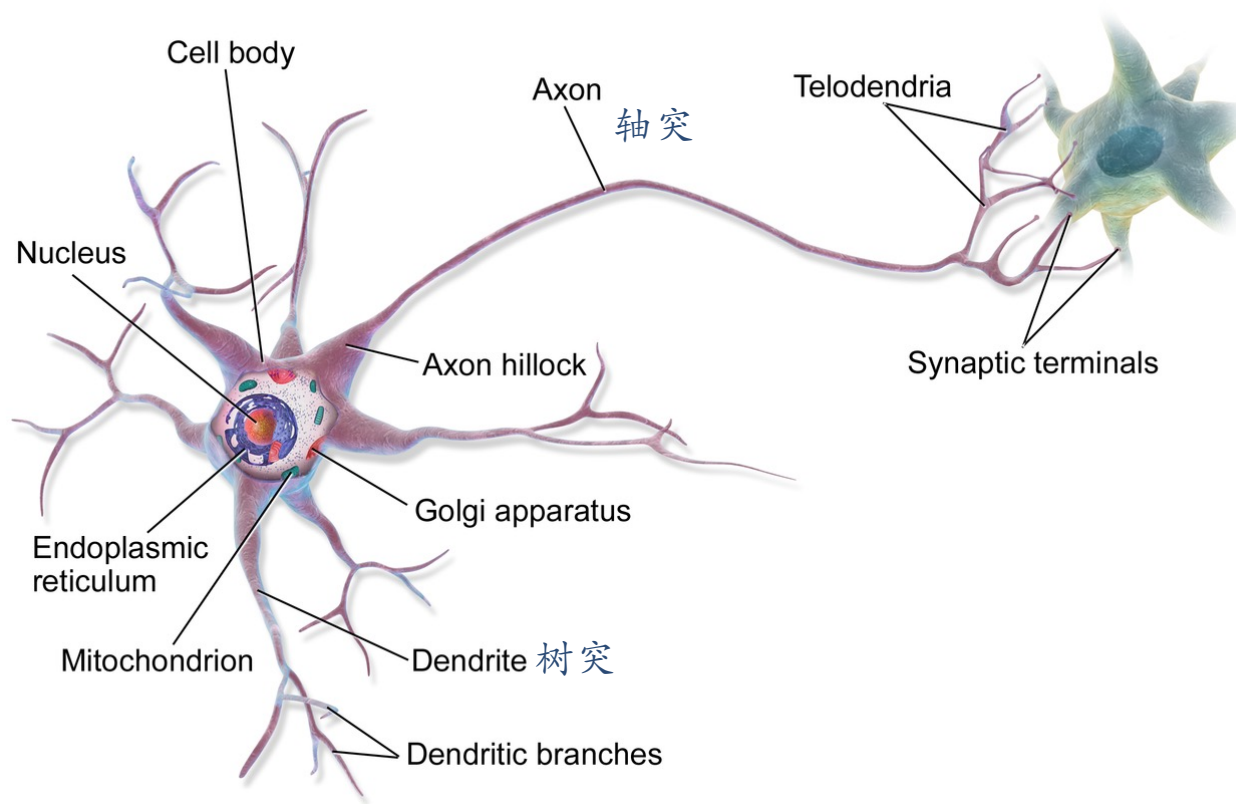
深度学习（**Deep Learning**）是一种以多层神经网络结构为基础，通过分层方式逐步提取特征表示，最终实现任务目标（如分类、识别、生成）的一种端到端的表示学习方法。

From Bengio 2013



神经网络

生物神经元 (Bio-Neuron)



神经网络如何学习？

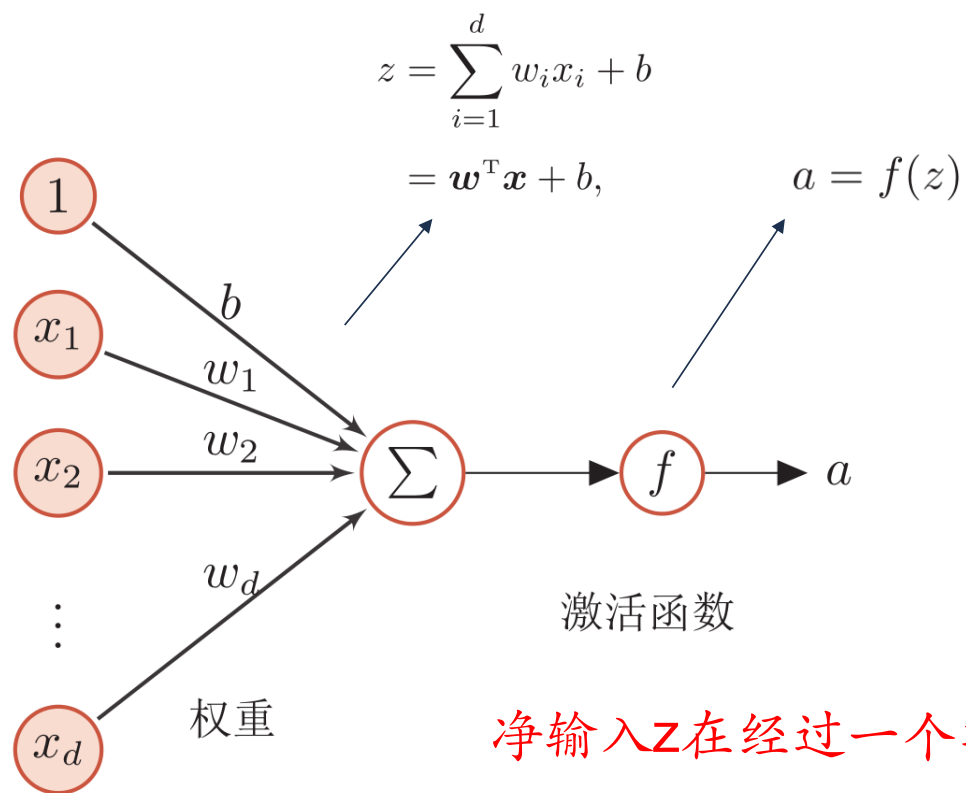
▶ 赫布法则 Hebb's Rule

- ▶ “当神经元 A 的一个轴突和神经元 B 很近，足以对它产生影响，并且持续地、重复地参与了对神经元 B 的兴奋，那么在这两个神经元或其中之一会发生某种生长过程或新陈代谢变化，以致于神经元 A 作为能使神经元 B 兴奋的细胞之一，它的效能加强了。”

----加拿大心理学家 Donald Hebb,
《行为的组织》，1949

- 人脑有两种记忆：长期记忆和短期记忆。短期记忆持续时间不超过一分钟。
如果一个经验重复足够的次数，此经验就可储存在长期记忆中。
- 短期记忆转化为长期记忆的过程就称为凝固作用。
- 人脑中的海马区为大脑结构凝固作用的核心区域。

人工神经元 (Artificial Neuron)



净输入 z 在经过一个非线性函数 $f(\cdot)$ 后，得到神经元的活性值 a
 $a=f(z)$

非线性函数 f 被称之为激活函数

激活函数 (Activation function) 的性质

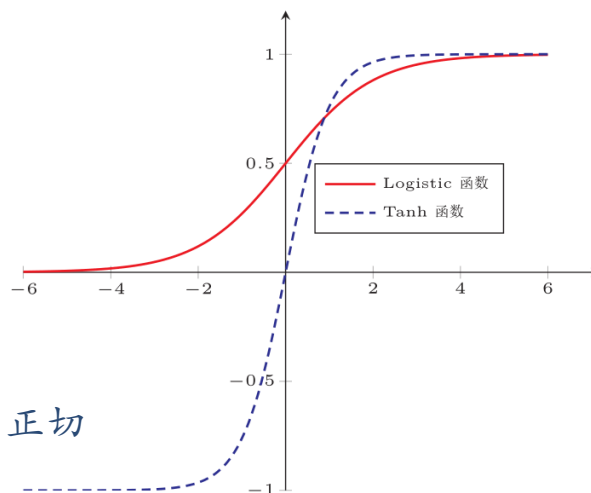
- ▶ 连续并可导（允许少数点上不可导）的非线性函数。
 - ▶ 可导的激活函数可以直接利用数值优化的方法来学习网络参数。
- ▶ 激活函数及其导函数要尽可能的简单
 - ▶ 有利于提高网络计算效率。
- ▶ 激活函数的导函数的值域要在一个合适的区间内
 - ▶ 不能太大也不能太小，否则会影响训练的效率和稳定性。

常见激活函数：S型函数

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)} \quad \text{双曲正切}$$

$$\tanh(x) = 2\sigma(2x) - 1$$



非零中心化的输出会使得其后的神经元的输入发生偏置偏移（**bias shift**），并进一步使得梯度下降的收敛速度变慢。

►性质：

►饱和函数

►Tanh函数是零中心化的，而logistic函数的输出恒大于0

常见激活函数: 斜坡函数

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \quad \text{Rectified Linear Unit (整流线性单元)}$$
$$= \max(0, x).$$

死亡ReLU问题 (Dying ReLU Problem)



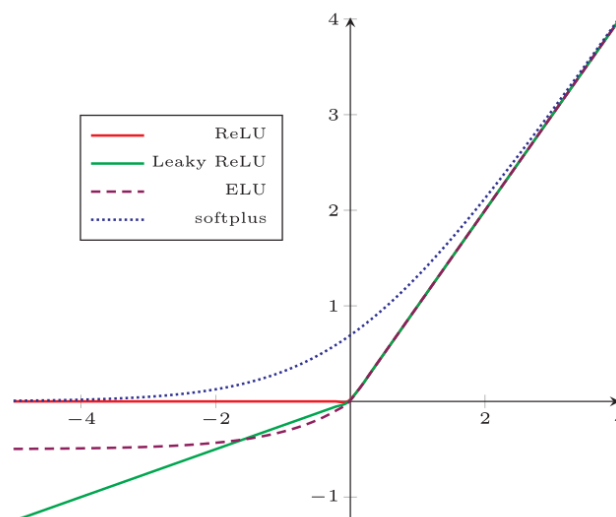
$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \gamma \min(0, x)$$

近似的零中心化的非线性函数

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

Relu函数的平滑版本

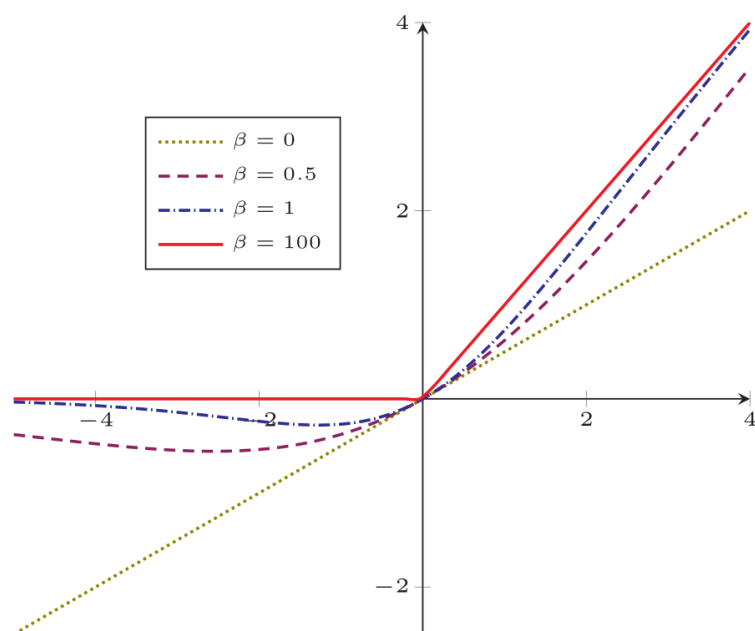
$$\text{softplus}(x) = \log(1 + \exp(x))$$



- ▶ 计算上更加高效
- ▶ 生物学合理性
 - ▶ 单侧抑制、宽兴奋边界
- ▶ 在一定程度上缓解梯度消失问题

常见激活函数: 复合函数

Swish函数 $\text{swish}(x) = x\sigma(\beta x)$



Google Brain 2017年提出，声称可替代 ReLU

来源与背景:

- 由 Google Brain 团队在 2017 年提出，作为 ReLU 的改进替代方案。
- 其中 $\sigma(\beta x)$ 是 sigmoid 函数， β 是可调参数。

特点:

- 当 $\beta = 0$ 时，Swish 退化为恒等函数 $f(x) = x$ ；
- β 越大，Swish 越接近于 ReLU；
- 与 ReLU 相比：Swish 是平滑的、非单调的，可以缓解“梯度消失”和“死亡 ReLU”问题；
- 可导、且梯度稳定，更适合深层网络。

常见激活函数及其导数

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

为了避免深度神经网络只作为一种深度线性分类器，必须要加入激活函数以希望其拥有非线性拟合的能力，其中，ReLU就是一个较好的激活函数。

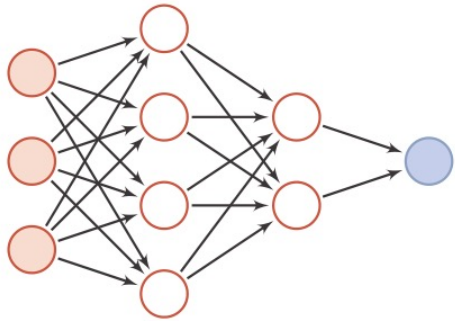
而同时为了避免其过拟合，又需要通过加入正则化来提高其泛化能力，其中，Dropout就是一种主流的正则化方式，而zoneout是Dropout的一个变种。

人工神经网络 (Artificial Neural Network)

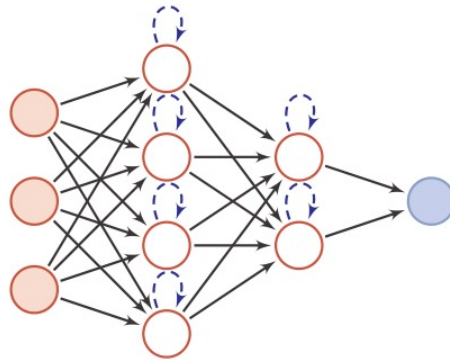
- ▶ 人工神经网络主要由大量的神经元以及它们之间的有向连接构成。因此考虑三方面：
- ▶ 神经元的激活规则
 - ▶ 主要是指神经元输入到输出之间的映射关系，一般为非线性函数。
- ▶ 网络的拓扑结构
 - ▶ 不同神经元之间的连接关系。
- ▶ 学习算法
 - ▶ 通过训练数据来学习神经网络的参数。

网络结构

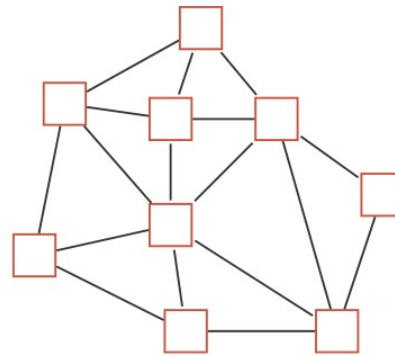
- ▶ 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。



(a) 前馈网络



(b) 记忆网络



(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

神经网络

- ▶ **神经网络是一种主要的连接主义模型**
- ▶ 20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理（Parallel Distributed Processing, PDP）网络,其有3个主要特征：
 - ▶ 1) 信息表示是分布式的（非局部的）；
 - ▶ 2) 记忆和知识是存储在单元之间的连接上；
 - ▶ 3) 通过逐渐改变单元之间的连接强度来学习新知识

引入误差反向传播来改进其学习能力之后，神经网络也越来越多应用在各种机器学习任务上。

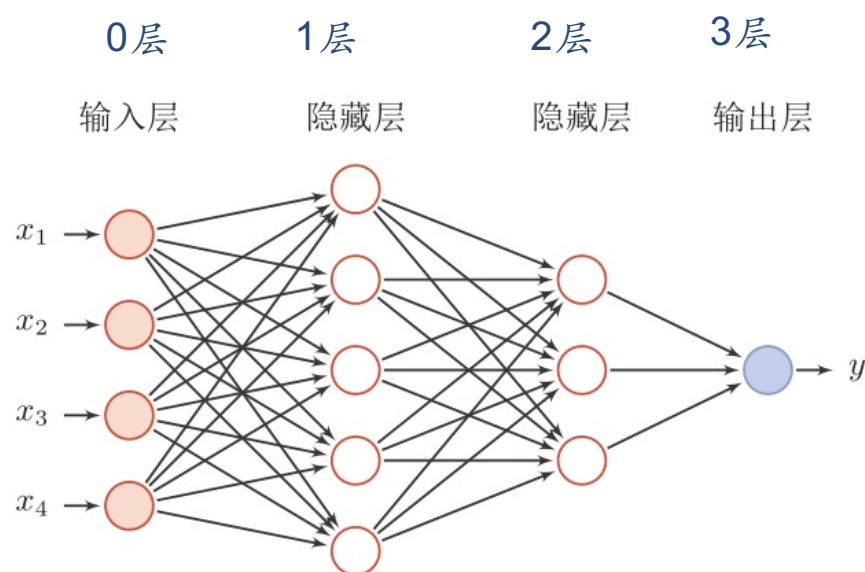


前馈神经网络

网络结构

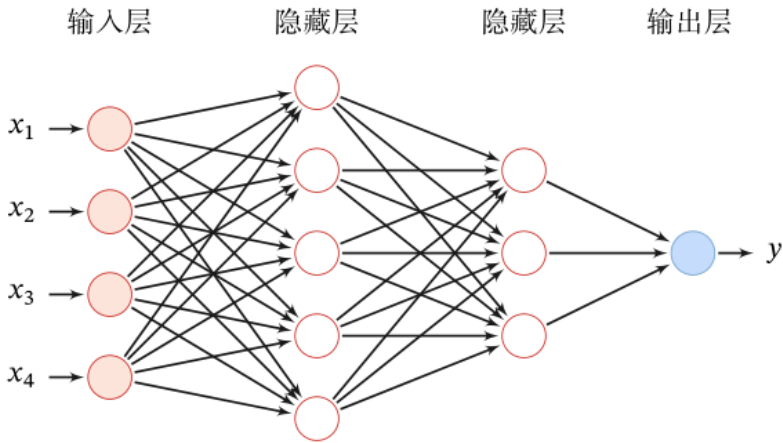
► 前馈神经网络（全连接神经网络、多层感知器）

- 各神经元分别属于不同的层，层内无连接。
- 相邻两层之间的神经元全部两两连接。
- 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。



前馈网络

给定一个前馈神经网络，用下面的记号来描述这样网络：



记号	含义	
L	神经网络的层数	超参数
M_l	第 l 层神经元的个数	
$f_l(\cdot)$	第 l 层神经元的激活函数	
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵	参数
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置	
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）	活性值
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）	

信息传递过程

► 前馈神经网络通过下面公式进行信息传播。

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

► 前馈计算：

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$

通用近似定理 (Universal Approximation Theorem)

定理 4.1 – 通用近似定理 (Universal Approximation Theorem)

[Cybenko, 1989, Hornik et al., 1989]: 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0, 1]^d$, $C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$, 存在一个整数 m , 和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$, $i = 1, \dots, m$, 以至于我们可以定义函数

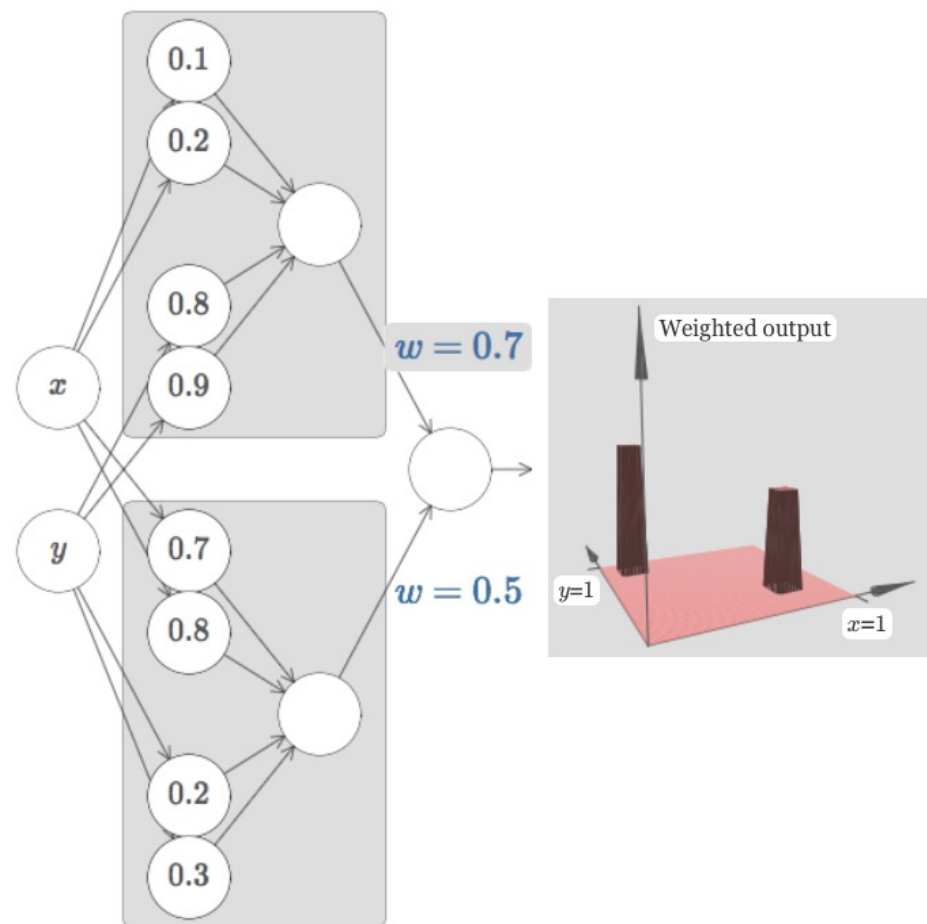
$$F(\mathbf{x}) = \sum_{i=1}^m v_i \varphi(\mathbf{w}_i^T \mathbf{x} + b_i), \quad (4.33)$$

作为函数 f 的近似实现, 即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \quad (4.34)$$

其中 $\epsilon > 0$ 是一个很小的正数。

根据通用近似定理, 对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络, 只要其隐藏层神经元的数量足够, 它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。



<http://neuralnetworksanddeeplearning.com/chap4.html>

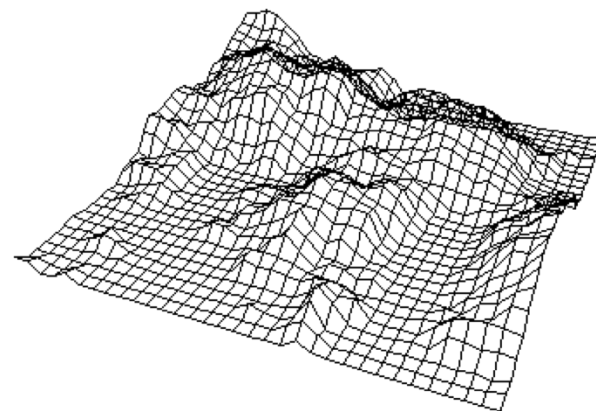
应用到深度学习

- ▶ 神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

$$\hat{y} = g(\varphi(\mathbf{x}), \theta)$$

分类器

神经网络



- ▶ 如果 $g(\cdot)$ 为Logistic回归，那么Logistic回归分类器可以看成神经网络的最后一层。
- ▶ 能够表示并不保证能够学习。参数众多，如何得到最优模型？

参数学习

► 对于多分类问题

- 如果使用Softmax回归分类器，相当于网络最后一层设置C个神经元，其输出经过Softmax函数进行归一化后可以作为每个类的条件概率。

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$$

- 采用交叉熵损失函数，对于样本(x,y)，其损失函数为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

参数学习

► 为什么使用交叉熵而不是MSE作为损失函数? 原因1: 最本质的原因 KL散度与交叉熵的关系

首先我们要明白MSE和交叉熵衡量的到底是什么。

对于MSE而言，其公式为：

$$\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

也就是真实值 - 预测值的平方和的平均值，放在坐标系中，也就是我们所熟知的欧氏距离，显然欧式距离越大，说明预测的越不准确，因此我们要最小化这一损失函数。

那么对于交叉熵而言，我们实际上是在衡量真实概率分布与预测概率分布之间的差异，因此我们真正在计算的实际上是KL散度，那么为什么使用交叉熵的公式呢，可以先看下KL散度公式：

$$D_{KL}(p||q) = \sum_{i=1}^n p(x_i) \log \frac{p(x_i)}{q(x_i)}$$

p 表示的是真实分布，q 表示的是预测分布

参数学习

► 为什么使用交叉熵而不是MSE作为损失函数? 原因1, 最本质的原因
KL散度与交叉熵的关系

现在我们将KL散度的公式拆开看一下:

$$\begin{aligned} D_{KL}(p||q) &= \sum_{i=1}^n p(x_i) \log \frac{p(x_i)}{q(x_i)} \\ &= \sum_{i=1}^n p(x_i) \log(p(x_i)) - \sum_{i=1}^n p(x_i) \log(q(x_i)) \end{aligned}$$

在其中我们看到了两项, 第一项是信息熵, 只与我们目前所知道的信息有关, 也就是只与我们用来训练的数据有关, 因此这一项是已知的常数。

$$\begin{aligned} D_{KL}(p||q) &= \sum_{i=1}^n p(x_i) \log(p(x_i)) - \sum_{i=1}^n p(x_i) \log(q(x_i)) \\ \Rightarrow D_{KL}(p||q) + \left(-\sum_{i=1}^n p(x_i) \log(p(x_i))\right) &= -\sum_{i=1}^n p(x_i) \log(q(x_i)) \end{aligned}$$

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

参数学习

►为什么使用交叉熵而不是MSE作为损失函数? 原因1, 最本质的原因
KL散度与交叉熵的关系

为什么Logistic回归使用交叉熵而不是MSE，从逻辑的角度出发，我们知道逻辑斯蒂回归的预测值是一个概率，而交叉熵又表示真是概率分布与预测概率分布的相似程度，因此选择使用交叉熵。

从MSE的角度来说，预测的概率与欧氏距离没有任何关系，并且在分类问题中，样本的值不存在大小关系，与欧氏距离更无关系，因此不适用MSE。MSE本质上等同于误差服从于高斯分布假设的极大似然估计，大部分分类问题的误差并不服从高斯分布。

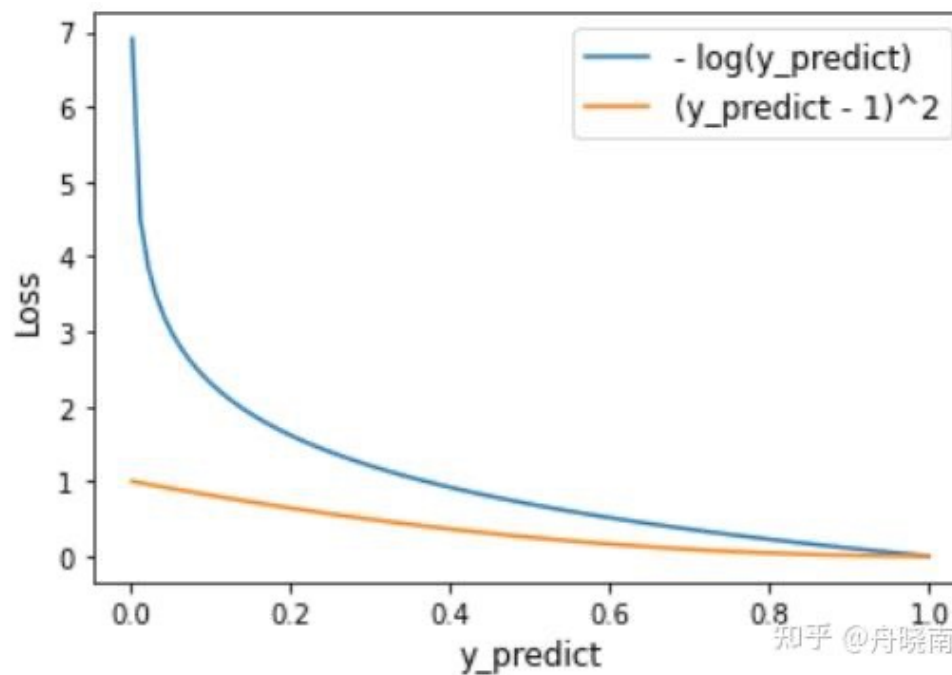
特征	传统模型概率输出	深度学习概率输出
概率来源	投票、拟合sigmoid、样本分布	softmax 输出
是否优化	通常不是直接优化概率分布	是，直接最小化交叉熵 (≈ KL散度)
连续可导	通常不可导	可导，适合梯度下降
适合的损失函数	MSE、Hinge loss	Cross-Entropy
是否一致使用交叉熵	✗ 并非主流	✔ 默认配置

参数学习

► 为什么使用交叉熵而不是MSE作为损失函数?

原因二：MSE 的损失小于交叉熵的损失，导致对分类错误的点的惩罚不够

假设 label 为 1，预测为 y_{predict} ，那么预测错误的点的 Loss 如下图所示，显然 log-loss 比 MSE 要大，并且对于预测错误的样本，错得越离谱，惩罚越严重。



参数学习

为什么使用交叉熵而不是MSE作为损失函数？

原因三：MSE 的梯度消失效应较大

如果我们使用 MSE 作为损失函数，那么来计算一下其梯度：

$$\begin{aligned} Loss &= (\hat{y}_i - y_i)^2 \\ \frac{\partial Loss}{\partial w} &= 2(\hat{y}_i - y_i) \frac{\partial (\hat{y}_i - y_i)}{\partial w} \\ &= 2(\hat{y}_i - y_i) \frac{\partial (\frac{1}{1+e^{-wx_i}})}{\partial w} \\ &= 2(\hat{y}_i - y_i) (\frac{1}{1+e^{-wx_i}})^2 \frac{\partial e^{-wx_i}}{\partial w} \\ &= 2(\hat{y}_i - y_i) (\frac{1}{1+e^{-wx_i}})^2 e^{-wx_i} \frac{\partial (-wx_i)}{\partial w} \\ &= 2(y_i - \hat{y}_i) (\frac{1}{1+e^{-wx_i}})^2 e^{-wx_i} x_i \end{aligned}$$

因为：

$$(\frac{1}{1+e^{-wx_i}})^2 e^{-wx_i} x_i = \hat{y}_i(1 - \hat{y}_i)$$

所以：

$$\frac{\partial Loss}{\partial w} = 2(y_i - \hat{y}_i)\hat{y}_i(1 - \hat{y}_i)x_i$$

我们可以看到在 MSE 的梯度中存在一项 $\hat{y}_i(1 - \hat{y}_i)$

也就是说，当我们的预测值接近于1或者接近于0时，会导致其梯度接近0，无法进行有效学习。当梯度很小的时候，应该减小步长（否则容易在最优解附近产生来回震荡），但是如果采用 MSE，当梯度很小的时候，无法知道是离目标很远还是已经在目标附近了。（离目标很近和离目标很远，其梯度都很小）。

而交叉熵不存在这一项，其梯度为 $(y_i - \hat{y}_i)x_i$ 即受误差的影响，所以当误差大的时候，权重更新就快，当误差小的时候，权重的更新就慢。

参数学习

为什么使用交叉熵而不是MSE作为损失函数?

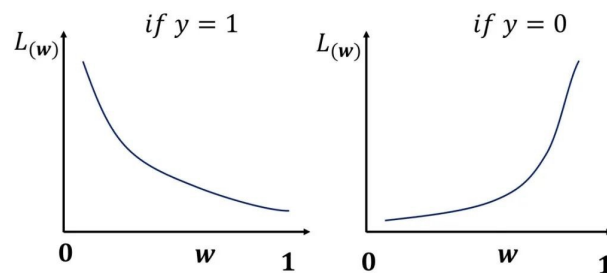
原因四：MSE 是非凸优化问题而交叉熵是凸优化问题

对MSE 一阶导数继续求二阶导

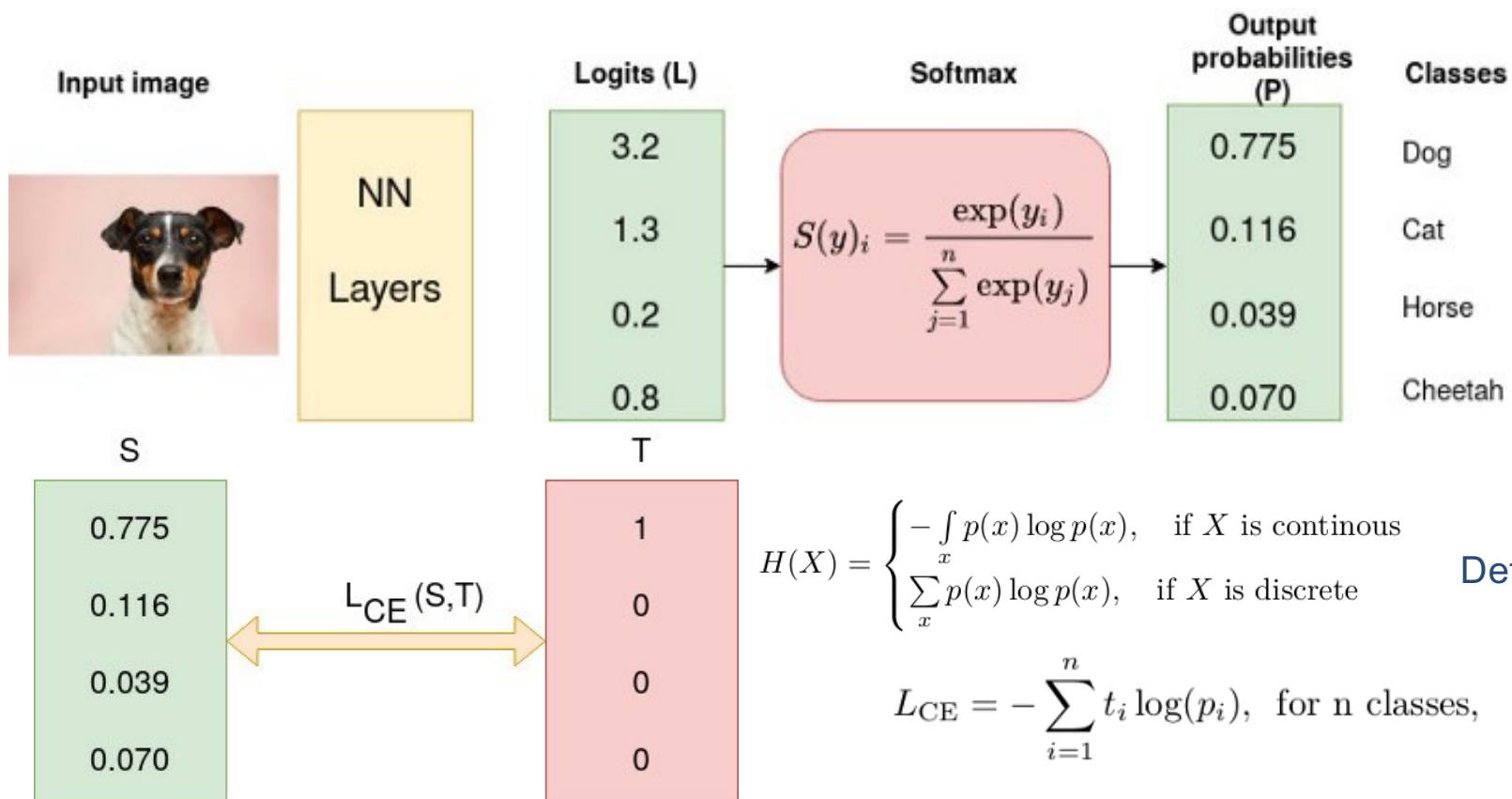
$$\begin{aligned} & \frac{\partial(y_i - \hat{y}_i)\hat{y}_i(1 - \hat{y}_i)x_i}{\partial w} \\ &= (-y_i x_i \hat{y}_i + (1 + y_i)x_i \hat{y}_i^2 - x_i \hat{y}_i^3)' \\ &= -y_i x_i^2 \hat{y}_i (1 - \hat{y}_i) + 2(1 + y_i)x_i^2 \hat{y}_i^2 (1 - \hat{y}_i) - 3x_i^2 \hat{y}_i^3 (1 - \hat{y}_i) \\ &= \hat{y}_i (1 - \hat{y}_i) x_i^2 (-y_i + 2(1 + y_i)\hat{y}_i - 3\hat{y}_i^2) \end{aligned}$$

其中 $\hat{y}_i(1 - \hat{y}_i)x_i^2$ 恒为正，只需要看 $-y_i + 2(1 + y_i)\hat{y}_i - 3\hat{y}_i^2$ 即可。

我们知道真实 label y_i 只能取1和0，当 $y_i=1$ 时，上式取值范围为 $(-4,3)$ ，当 $y_i=0$ 时，上式取值范围为 $(-3,2)$ ，因此二阶导不恒大于等于0，因此MSE损失函数为非凸函数。



交叉熵损失函数 (Cross-Entropy Function)



$$H(X) = \begin{cases} - \int_x p(x) \log p(x), & \text{if } X \text{ is continuous} \\ \sum_x p(x) \log p(x), & \text{if } X \text{ is discrete} \end{cases}$$

Definition of Entropy

$$L_{CE} = - \sum_{i=1}^n t_i \log(p_i), \text{ for } n \text{ classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the i^{th} class.

参数学习

- ▶ 给定训练集为 $D = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ ，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络，得到网络输出为 $\hat{y}^{(n)}$ ，其在数据集 D 上的结构化风险函数为：

$$\mathcal{R}(W, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|W\|_F^2$$

- ▶ 梯度下降

$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial W^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$

梯度下降

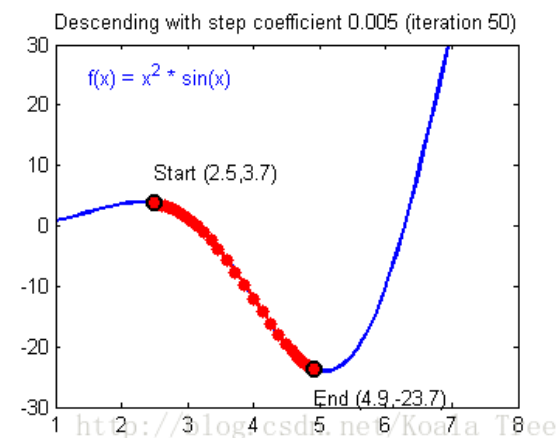
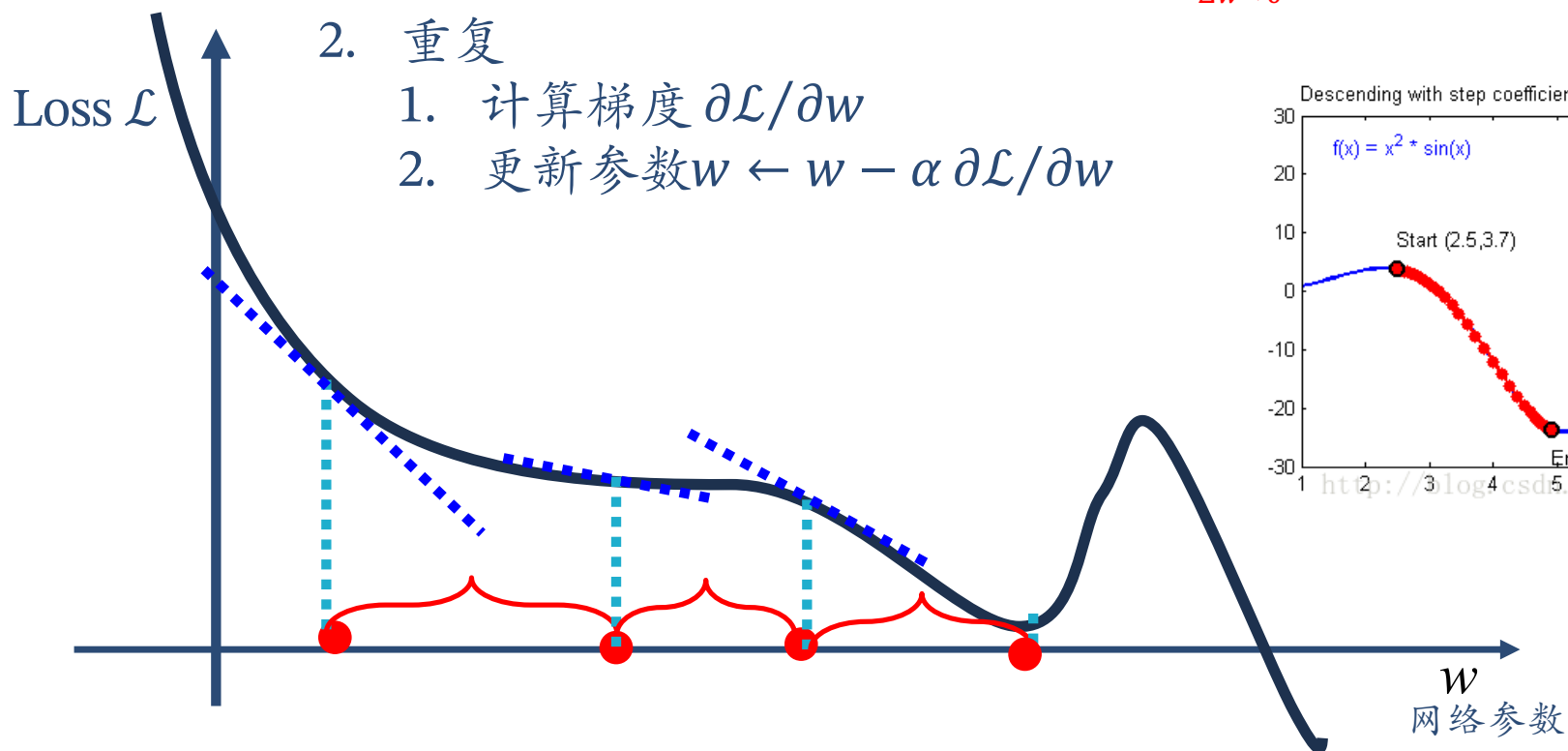
1. 初始化 w

2. 重复

1. 计算梯度 $\partial \mathcal{L} / \partial w$

2. 更新参数 $w \leftarrow w - \alpha \partial \mathcal{L} / \partial w$

$$\text{梯度: } \frac{\partial f(w)}{\partial w} = \lim_{\Delta w \rightarrow 0} \frac{f(w + \Delta w) - f(w)}{\Delta w}$$



反向传播算法 (Backpropagation)

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{m^{(l)}}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_i^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= [0, \dots, a_j^{(l-1)}, \dots, 0] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}} \quad \text{误差项}$$

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

反向传播算法 (Backpropagation)

计算 $\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (W^{(l+1)})^T$$

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

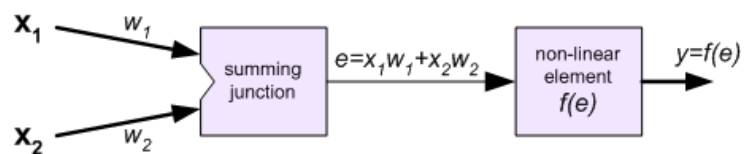
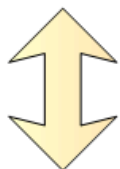
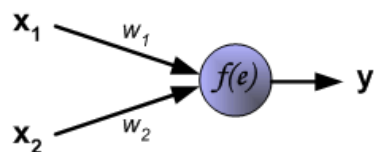
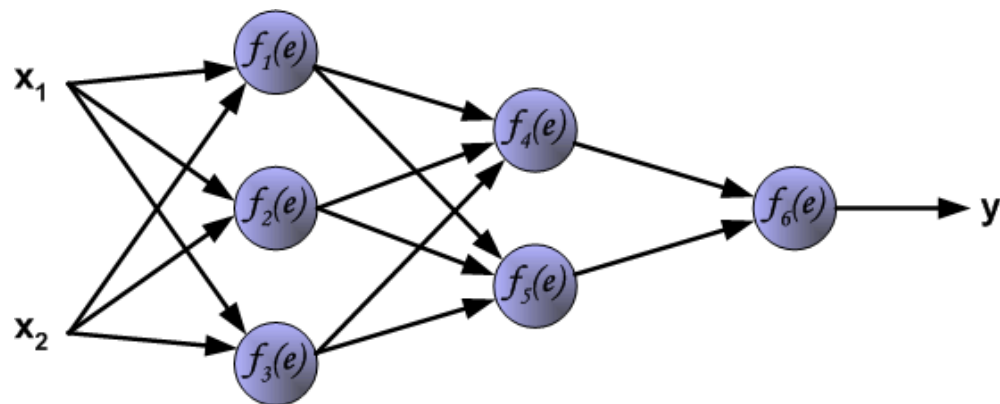
$$\begin{aligned} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} &= \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \end{aligned}$$

$$\begin{aligned} &= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (W^{(l+1)})^T \cdot \delta^{(l+1)} \\ &= f'_l(\mathbf{z}^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)}), \end{aligned}$$

反向传播算法 (Backpropagation)

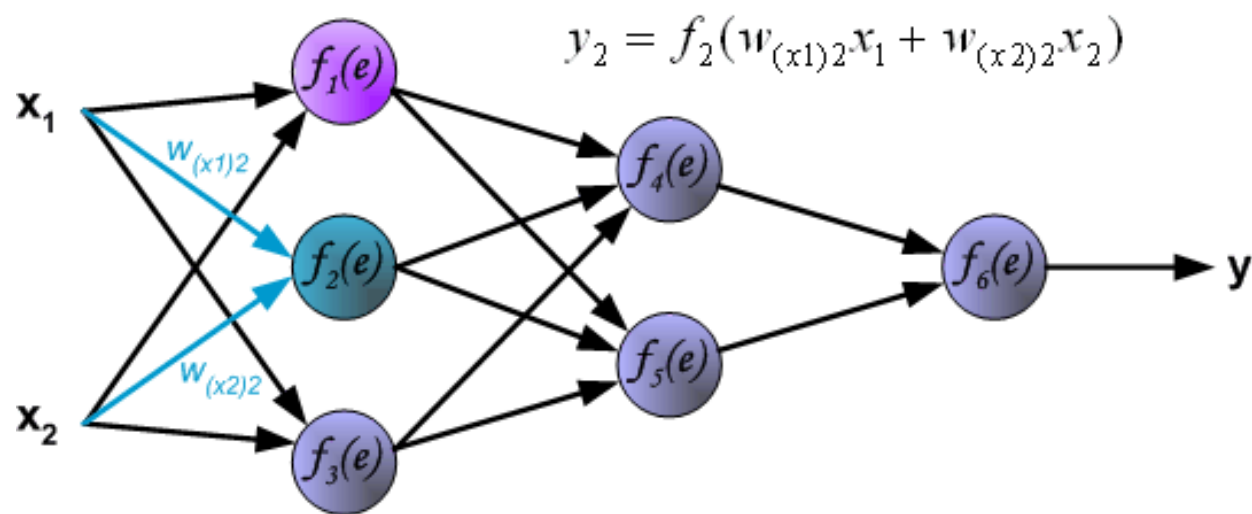
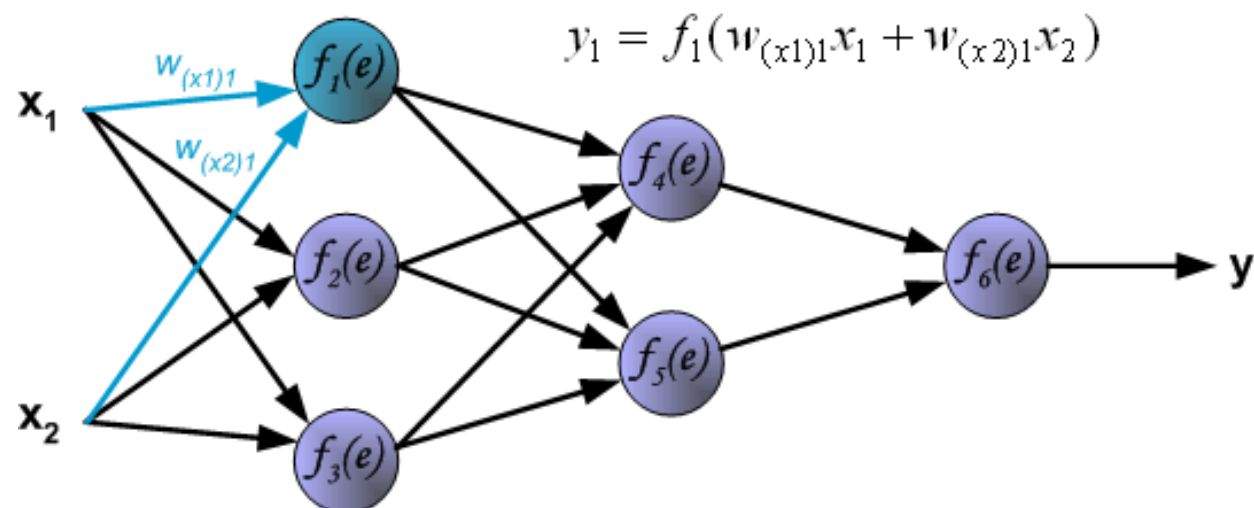
- ▶ 前馈神经网络的训练过程可以分为以下三步
 - ▶ 前向计算每一层的状态和激活值，直到最后一层
 - ▶ 反向计算每一层的误差项
 - ▶ 计算每一层的偏导数，更新参数

反向传播算法 (Backpropagation)

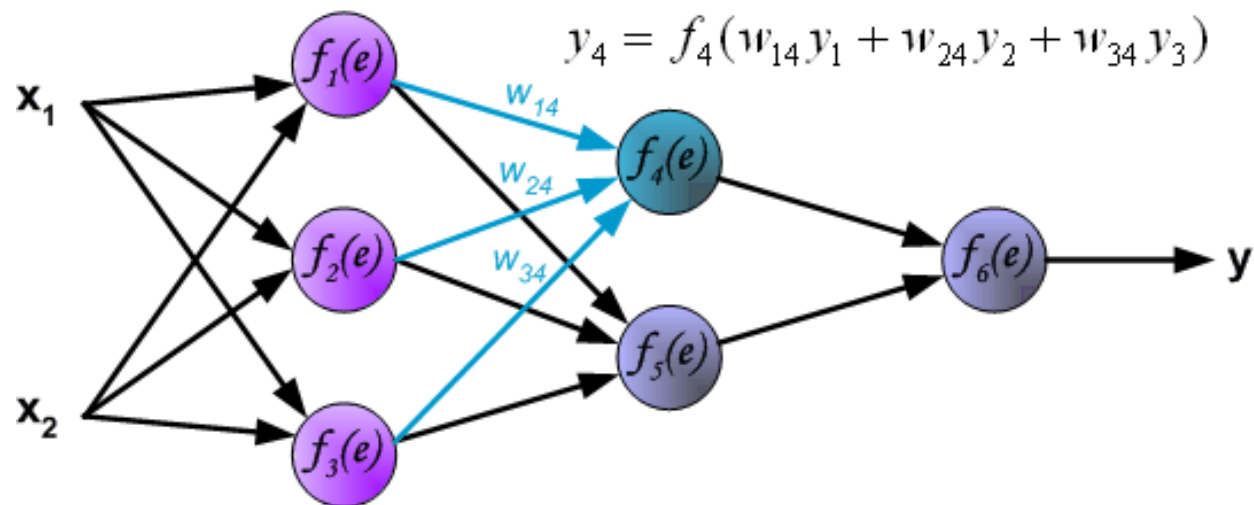
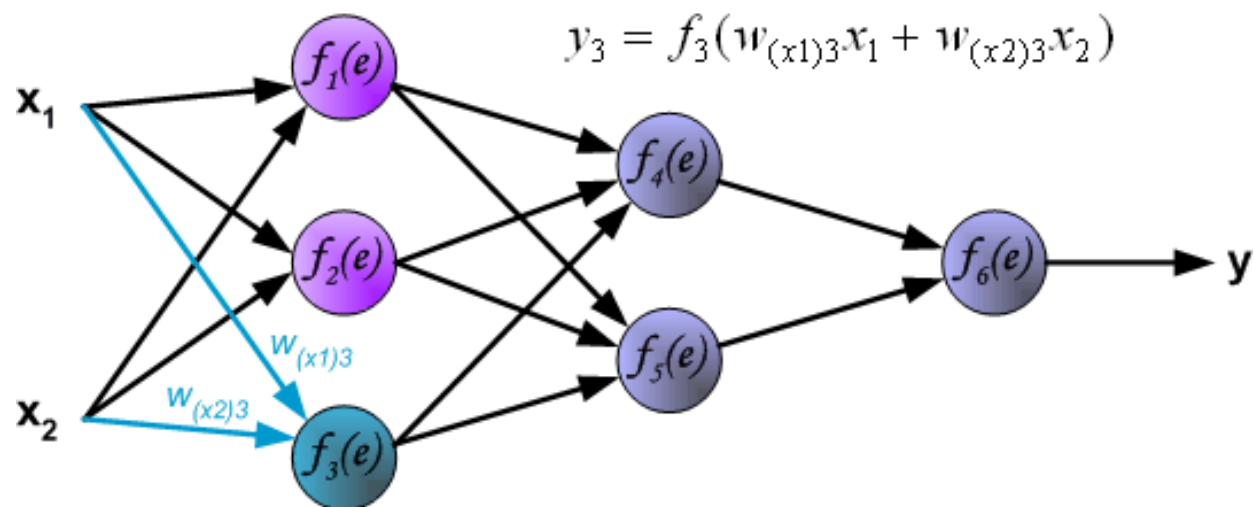


http://galaxy.agh.edu.pl/%7Evlsi/AI/backp_t_en/backprop.html

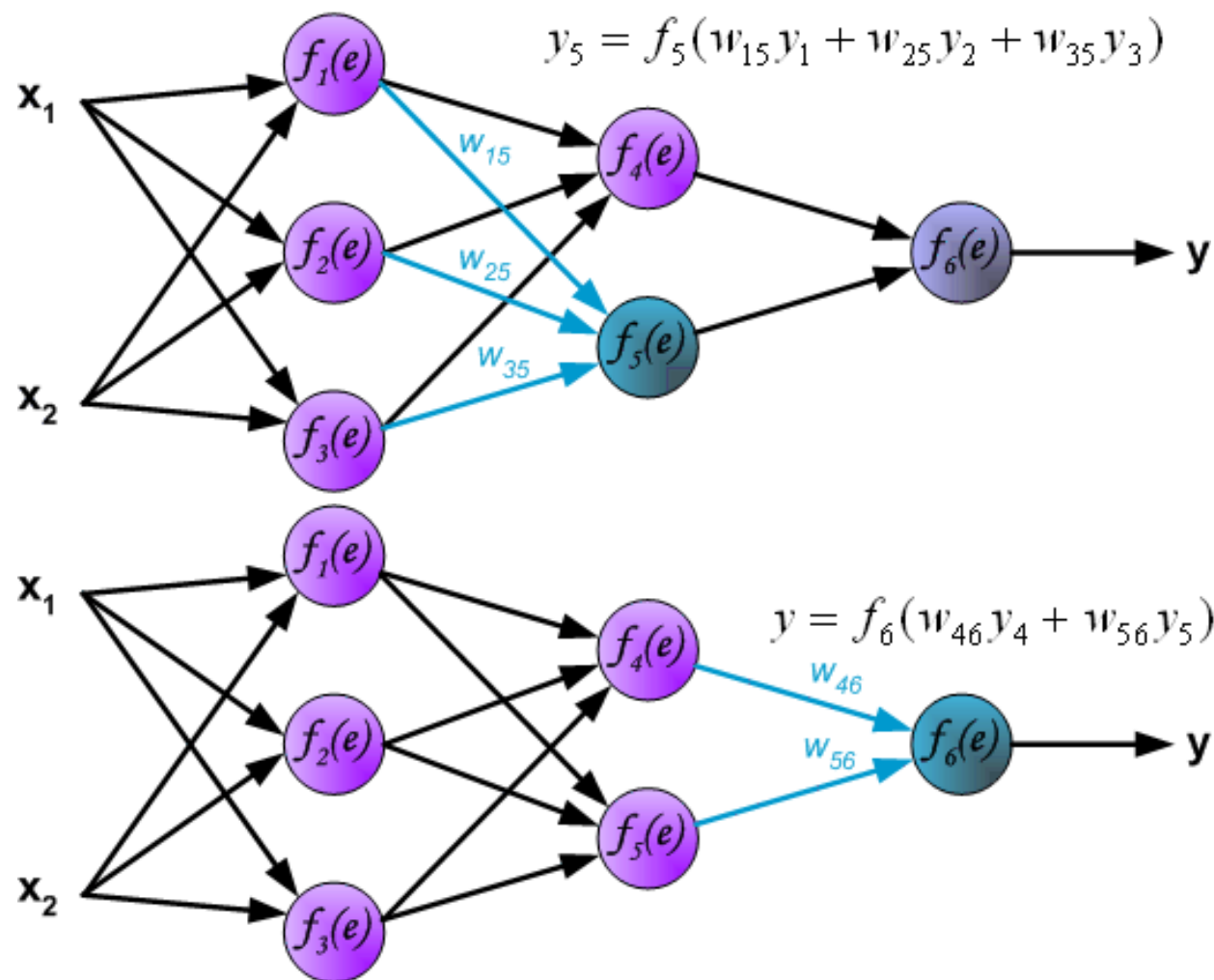
反向传播算法 (Backpropagation, Step-1)



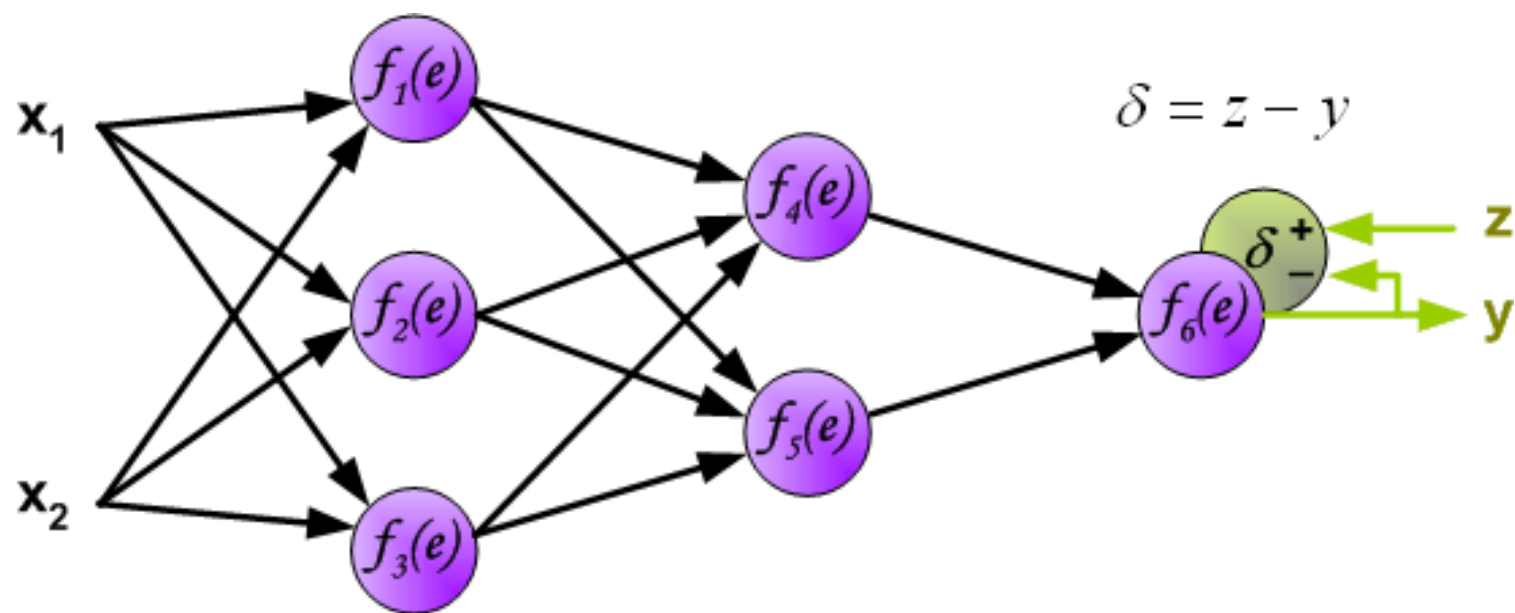
反向传播算法 (Backpropagation, Step-1)



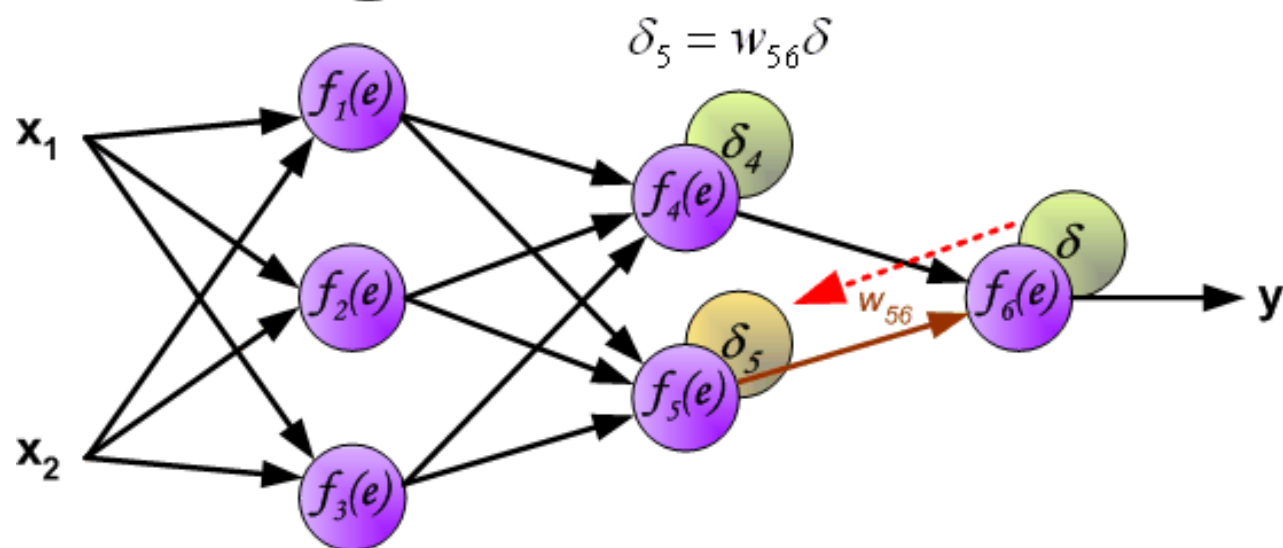
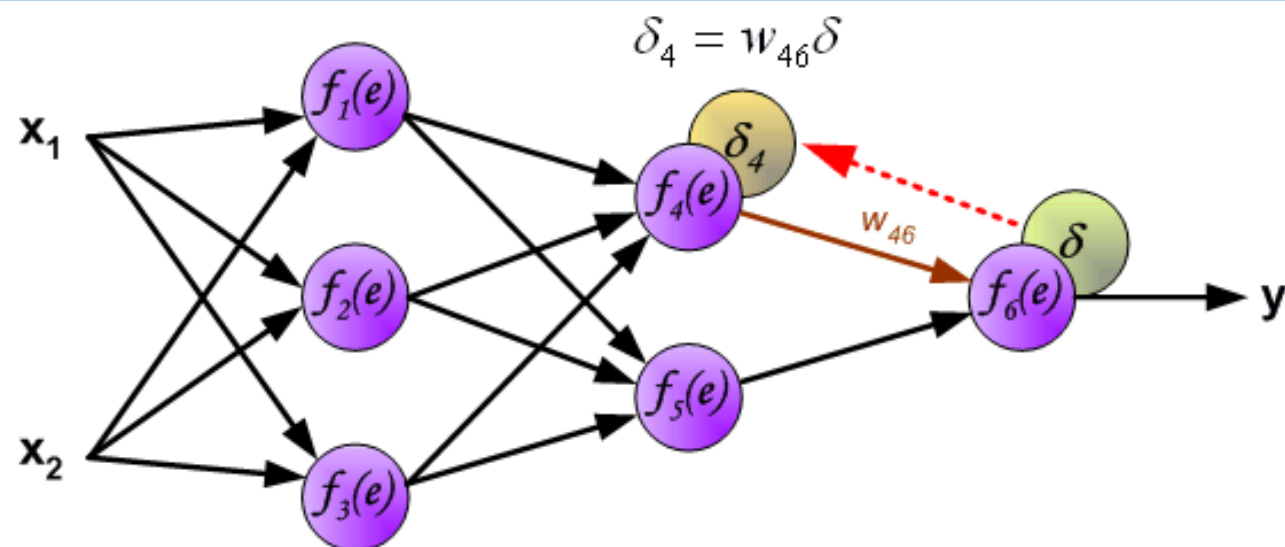
反向传播算法 (Backpropagation, Step-1)



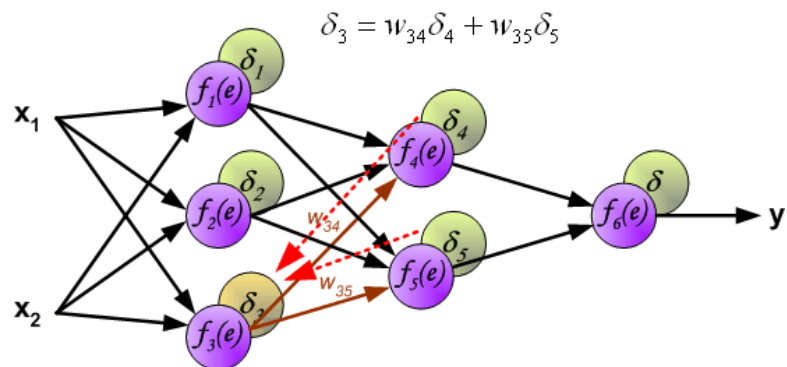
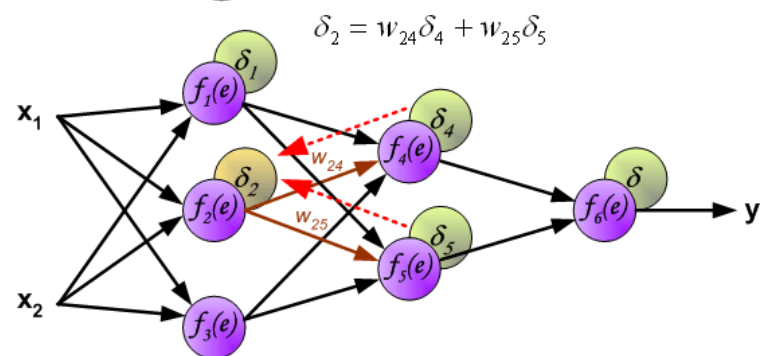
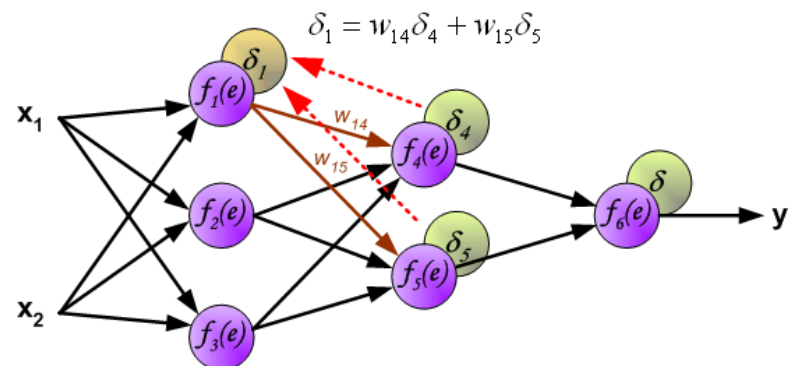
反向传播算法 (Backpropagation, Step-1)



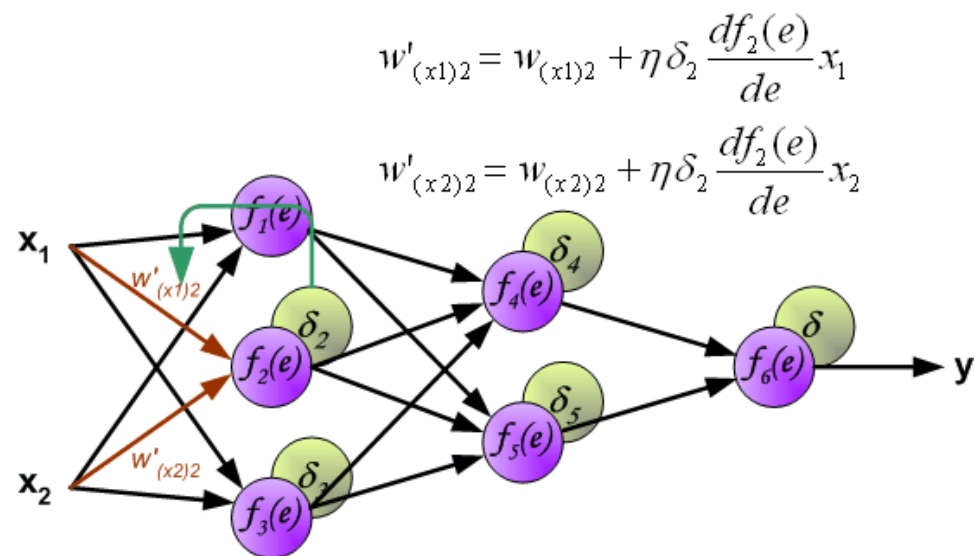
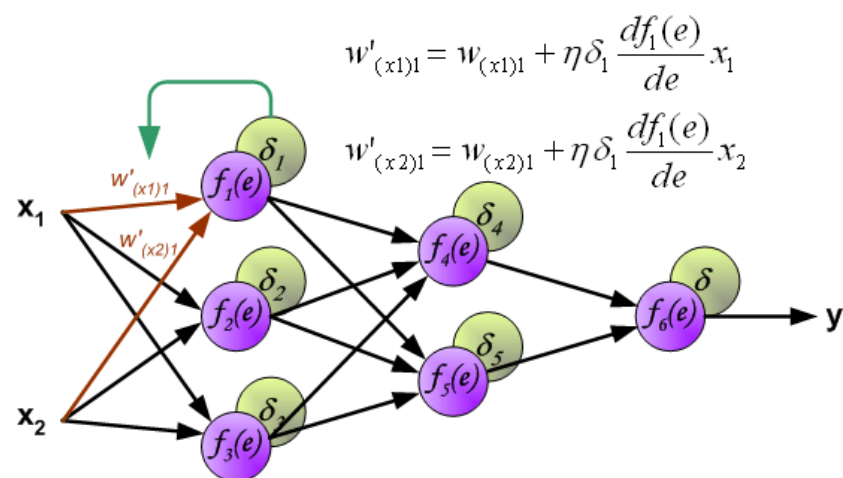
反向传播算法 (Backpropagation, Step-2)



反向传播算法 (Backpropagation, Step-2)



反向传播算法 (Backpropagation, Step-3)



如何计算梯度?

▶神经网络为一个复杂的复合函数

▶链式法则

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

▶反向传播算法

▶根据前馈网络的特点而设计的高效方法

▶一个更加通用的计算方法

▶自动微分 (Automatic Differentiation, AD)

矩阵微积分

▶ 矩阵微积分 (Matrix Calculus) 是多元微积分的一种表达方式, 即使用矩阵和向量来表示因变量每个成分关于自变量每个成分的偏导数。

▶ 分母布局

▶ 标量关于向量的偏导数

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_p} \right]^T$$

▶ 向量关于向量的偏导数

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_q}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_p} & \dots & \frac{\partial y_q}{\partial x_p} \end{bmatrix} \in \mathbb{R}^{p \times q}$$

链式法则

► 链式法则 (Chain Rule) 是在微积分中求复合函数导数的一种常用方法。

(1) 若 $x \in \mathbb{R}$, $\mathbf{u} = u(x) \in \mathbb{R}^s$, $\mathbf{g} = g(\mathbf{u}) \in \mathbb{R}^t$, 则

标量

$$\frac{\partial \mathbf{g}}{\partial x} = \frac{\partial \mathbf{u}}{\partial x} \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \in \mathbb{R}^{1 \times t}.$$

(2) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = g(\mathbf{x}) \in \mathbb{R}^s$, $\mathbf{z} = f(\mathbf{y}) \in \mathbb{R}^t$, 则

向量

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \in \mathbb{R}^{p \times t}.$$

(3) 若 $X \in \mathbb{R}^{p \times q}$ 为矩阵, $\mathbf{y} = g(X) \in \mathbb{R}^s$, $z = f(\mathbf{y}) \in \mathbb{R}$, 则

矩阵

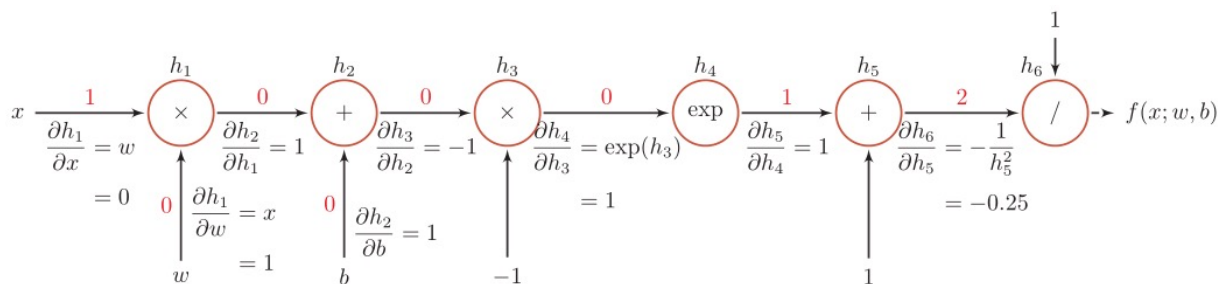
$$\frac{\partial z}{\partial X_{ij}} = \frac{\partial \mathbf{y}}{\partial X_{ij}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}.$$

计算图与自动微分

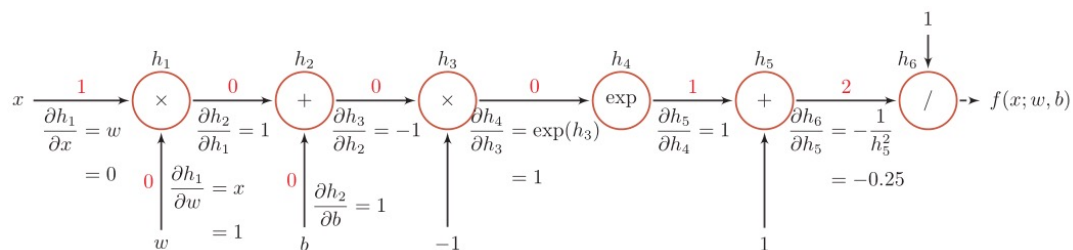
- 自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$

- 计算图



计算图



函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

当 $x = 1, w = 0, b = 0$ 时，可以得到

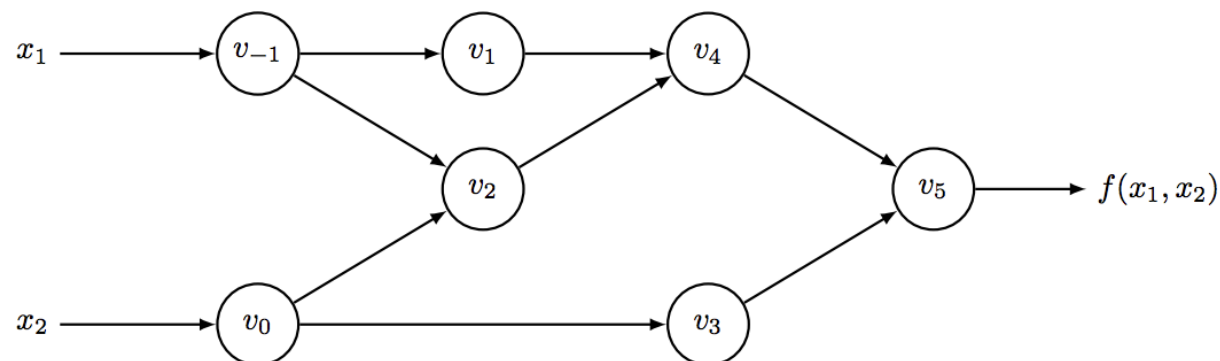
$$\begin{aligned}
 \frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\
 &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\
 &= 0.25.
 \end{aligned}$$

计算图

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$

自动微分法Forward Mode

- 所有数值计算归根结底是一系列有限的可微算子的组合



<http://blog.csdn.net/aws321715>

Forward Evaluation Trace		
v_{-1}	$= x_1$	$= 2$
v_0	$= x_2$	$= 5$
v_1	$= \ln v_{-1}$	$= \ln 2$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$
v_3	$= \sin v_0$	$= \sin 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$
y	$= v_5$	$= 11.652$

Forward Derivative Trace		
$\dot{v}_{-1}$	$= \dot{x}_1$	$= 1$
$\dot{v}_0$	$= \dot{x}_2$	$= 0$
$\dot{v}_1$	$= \dot{v}_{-1} / v_{-1}$	$= 1/2$
$\dot{v}_2$	$= \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1}$	$= 1 \times 5 + 0 \times 2$
$\dot{v}_3$	$= \dot{v}_0 \times \cos v_0$	$= 0 \times \cos 5$
$\dot{v}_4$	$= \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$
$\dot{v}_5$	$= \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$
$\dot{y}$	$= \dot{v}_5$	$= 5.5$

考虑n个输入，求解函数梯度需要n遍如上计算过程

<http://blog.csdn.net/aws3217150>

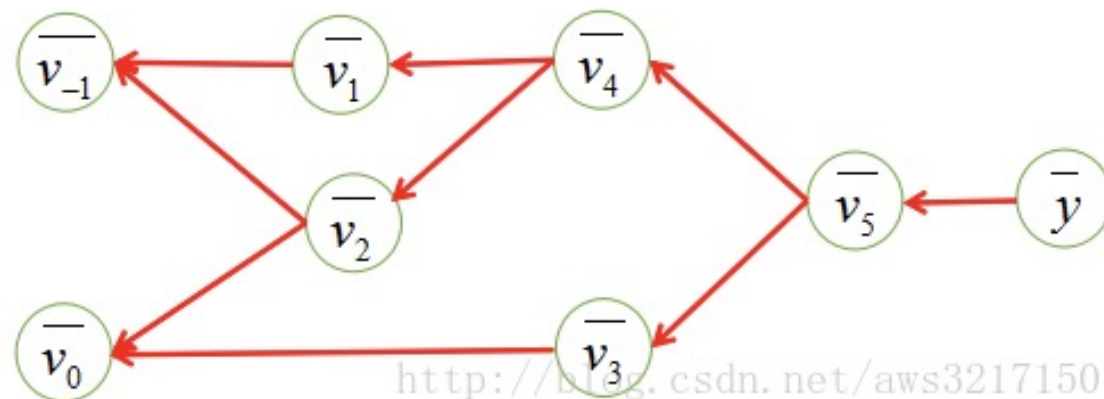
左半部分是从左往右每个图节点的求值结果，右半部分是每个节点对于 x_1 的求导结果

计算图

自动微分法 Backward Mode

- 所有数值计算归根结底是一系列有限的可微算子的组合

Forward Evaluation Trace	Reverse Adjoint Trace
$v_{-1} = x_1 = 2$	$\bar{x}_1 = \bar{v}_{-1} = 5.5$
$v_0 = x_2 = 5$	$\bar{x}_2 = \bar{v}_0 = 1.716$
$v_1 = \ln v_{-1} = \ln 2$	$\bar{v}_{-1} = \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} = \bar{v}_{-1} + \bar{v}_1 / v_{-1} = 5.5$
$v_2 = v_{-1} \times v_0 = 2 \times 5$	$\bar{v}_0 = \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0} = \bar{v}_0 + \bar{v}_2 \times v_{-1} = 1.716$
$v_3 = \sin v_0 = \sin 5$	$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_2 \times v_0 = 5$
$v_4 = v_1 + v_2 = 0.693 + 10$	$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \times \cos v_0 = -0.284$
$v_5 = v_4 - v_3 = 10.693 + 0.959$	$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$
$y = v_5 = 11.652$	$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$
	$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$
	$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$
	$\bar{v}_5 = \bar{y} = 1$



对于节点v3

$$\frac{dy}{dv_3} = \frac{dy}{dv_5} \frac{dv_5}{dv_3} \quad \Rightarrow \quad \frac{dy}{dv_3} = \bar{v}_5 \frac{dv_5}{dv_3}$$

$$\frac{dy}{dv_0} = \frac{dy}{dv_2} \frac{dv_2}{dv_0} + \frac{dy}{dv_3} \frac{dv_3}{dv_0}$$

$$\frac{dy}{dv_0} = \bar{v}_2 \frac{dv_2}{dv_0} + \bar{v}_3 \frac{dv_3}{dv_0}$$

只需要一遍reverse mode的计算过程，便可以求出输出对于各个输入的导数，从而轻松求取梯度

自动微分

- ▶ 前向模式和反向模式

- ▶ 反向模式和反向传播的计算梯度的方式相同

- ▶ 如果函数和参数之间有多条路径，可以将这多条路径上的导数再进行相加，得到最终的梯度。

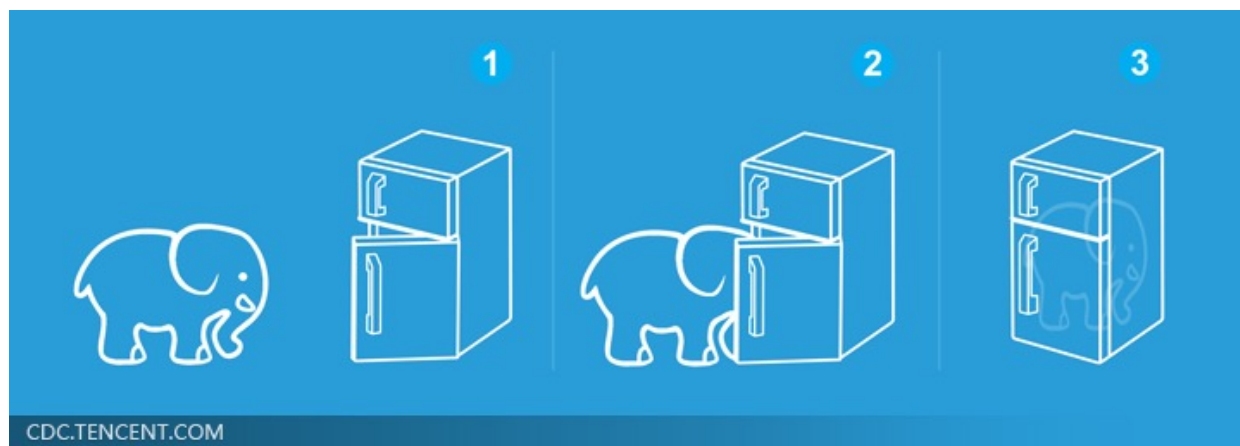
静态计算图和动态计算图

- ▶ 静态计算图是在编译时构建计算图，计算图构建好之后在程序运行时不能改变。
 - ▶ Theano和Tensorflow
- ▶ 动态计算图是在程序运行时动态构建。两种构建方式各有优缺点。
 - ▶ DyNet, Chainer和PyTorch
- ▶ 静态计算图在构建时可以进行优化，并行能力强，但灵活性比较低。动态计算图则不容易优化，当不同输入的网络结构不一致时，难以并行计算，但是灵活性比较高。

深度学习的三个步骤



Deep Learning is so simple



如何实现?

Deep Learning



What society thinks I do



What my friends think I do



What other computer scientists think I do



What mathematicians think I do



What I think I do

```
from theano import *
```

What I actually do

常用的深度学习框架

- ▶ 简易和快速的原型设计
- ▶ 自动梯度计算
- ▶ 无缝CPU和GPU切换



 PaddlePaddle



常用的深度学习框架

```
from keras.models import Sequential
from keras.layers import Dense, Activation
from keras.optimizers import SGD

model = Sequential()
model.add(Dense(output_dim=64, input_dim=100))
model.add(Activation("relu"))
model.add(Dense(output_dim=10))
model.add(Activation("softmax"))

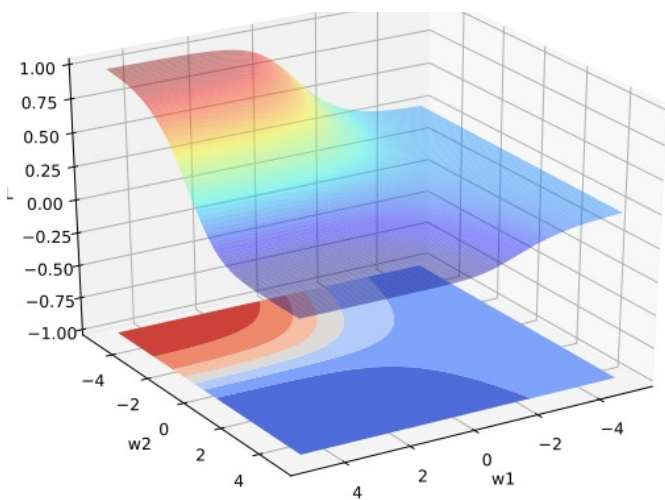
model.compile(loss='categorical_crossentropy',
              optimizer='sgd', metrics=['accuracy'])

model.fit(X_train, Y_train, nb_epoch=5, batch_size=32)

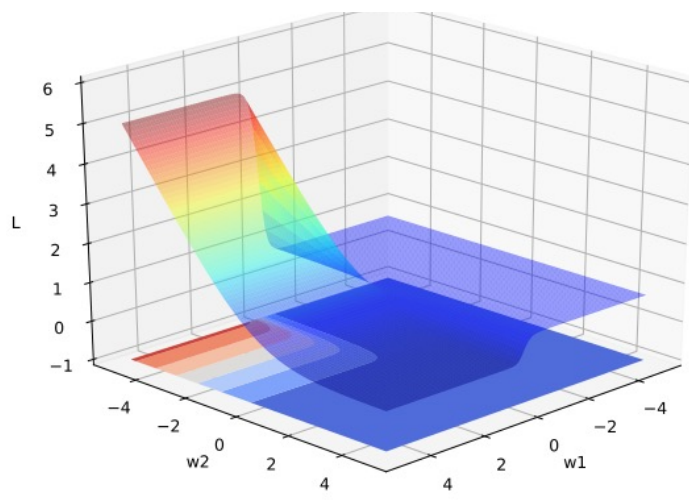
loss = model.evaluate(X_test, Y_test, batch_size=32)
```

优化问题

► 非凸优化问题



(a) 平方误差损失



(b) 交叉熵损失

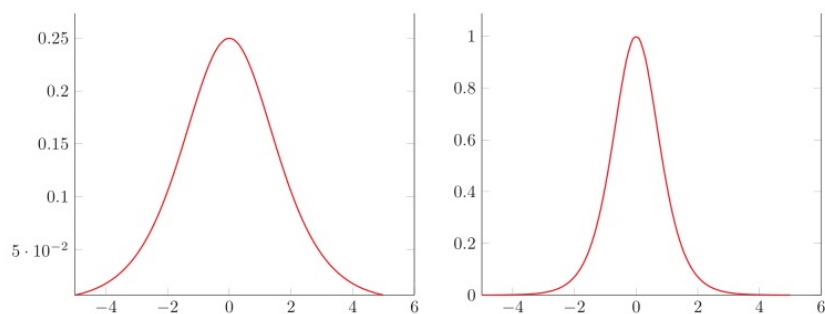
图 4.9 神经网络 $y = \sigma(w_2\sigma(w_1x))$ 的损失函数

优化问题

► 梯度消失问题 (Vanishing Gradient Problem)

$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$



(a) logistic 函数的导数

(b) tanh 函数的导数

优化问题

▶ 难点

- ▶ 参数过多，影响训练
- ▶ 非凸优化问题：即存在局部最优而非全局最优解，影响迭代
- ▶ 梯度消失问题，下层参数比较难调
- ▶ 参数解释起来比较困难

▶ 需求

- ▶ 计算资源要大
- ▶ 数据要多
- ▶ 算法效率要好：即收敛快

► 知识点

- 激活函数
- 误差反向传播

► 编程练习

Simple neural network

从零开始实现一个不依赖深度学习框架的两层前馈神经网络，完成对二维数据的二分类任务。

该神经网络结构如下：

- 输入层：2维特征（如 $[x_1, x_2]$ ）
- 隐藏层：4 个神经元，使用 Sigmoid 激活
- 输出层：1 个神经元，输出值使用 Sigmoid 激活（表示为正类的概率）

